

Laporan Tugas Kecil 3
IF2211 - Strategi Algoritma
Ice Sliding Puzzle Solver



Disusun oleh:

Nazwan Siddqi Muttaqin 18223066

Matthew Sebastian Kurniawan 18223096

Program Studi Teknik Informatika
Sekolah Teknik Elektro dan Informatika
Institut Teknologi Bandung

2026

DAFTAR ISI

DAFTAR ISI.....	2
A. Pendahuluan.....	3
B. Implementasi Algoritma.....	4
a. Representasi Status Permainan.....	4
b. Mekanisma Successor Generation.....	4
c. Implementasi Unified Best-First Search (UCS, GBFS, A*).....	6
d. Algoritma Alternatif: Iterative Deepening A* (IDA*) (Bonus).....	9
e. Implementasi Fungsi Heuristik.....	13
C. Analisis Strategi Algoritma.....	16
a. Definisi Fungsi Evaluasi $f(n)$ dan $g(n)$	16
b. Pembuktian Admissibility Heuristik A*.....	16
c. Perbandingan Teoritis.....	18
D. Source Code.....	19
E. Hasil Program.....	74
a. Tangkapan Layar Percobaan.....	74
b. Analisis Hasil Percobaan.....	81
F. Link Repository.....	84
G. Surat Pernyataan.....	84
H. Lampiran.....	84
I. Referensi.....	85

A. Pendahuluan

Ice Sliding Puzzle merupakan permainan logika di mana pemain ditugaskan untuk mengarahkan sebuah karakter dari titik awal penelusuran menuju titik keluar di atas representasi permukaan es yang licin. Permainan ini memiliki mekanika pergerakan khusus di mana karakter hanya dapat bergerak secara garis lurus, baik horizontal maupun vertikal. Karena sifat permukaan yang licin, karakter tidak akan berhenti meluncur sampai jalurnya terhalang oleh sebuah rintangan atau dinding pembatas area permainan.

Dalam penelusurannya, terdapat berbagai elemen pada peta permainan yang menentukan validitas pergerakan. Rintangan atau batu bertindak sebagai titik henti yang aman bagi karakter. Sebaliknya, terdapat elemen lava yang mengakibatkan status game over apabila dilewati. Selain itu, pergerakan karakter yang meluncur keluar dari batas papan permainan tanpa menabrak rintangan juga dikategorikan sebagai kegagalan.

Tingkat kompleksitas permasalahan ini ditambah dengan adanya aturan pengurutan capaian objektif. Di atas papan permainan, terdapat sejumlah petak bernomor (0 hingga maksimal 9) yang diwajibkan untuk dilewati oleh karakter sesuai dengan urutan numeriknya. Elemen nomor yang lebih besar tidak boleh diinjak sebelum elemen dengan nomor sebelumnya berhasil dilewati. Karakter baru dinyatakan sukses apabila berhasil berhenti tepat di titik keluar setelah seluruh petak angka divalidasi sesuai urutan. Setiap pergerakan meluncur ini dievaluasi berdasarkan akumulasi biaya (cost) dari setiap petak yang dilewati.

Pada Tugas Kecil 3 Strategi Algoritma ini, mahasiswa diminta untuk merancang dan membangun sebuah program yang mampu menelusuri dan menemukan rute solusi yang sah dari permainan tersebut. Pencarian rute diimplementasikan menggunakan algoritma pathfinding graf, yaitu Uniform Cost Search (UCS), Greedy Best-First Search (GBFS), dan algoritma A-Star (A^*). Algoritma tersebut digunakan untuk mengeksplorasi pohon kemungkinan status pergerakan karakter guna menghasilkan solusi dengan efisiensi biaya yang paling optimal.

B. Implementasi Algoritma

a. Representasi Status Permainan

Dalam memodelkan persoalan *Ice Sliding Puzzle*, setiap status (simpul) di dalam ruang pencarian direpresentasikan oleh tiga elemen utama: posisi absolut karakter pada sumbu X, posisi absolut pada sumbu Y, dan target angka selanjutnya (**NextDigit**) yang wajib dicapai oleh karakter. Papan permainan dimodelkan sebagai matriks dua dimensi yang menyimpan informasi jenis petak (es, batu, lava, angka, atau tujuan) beserta nilai biaya untuk melewatinya.

b. Mekanisma Successor Generation

Fungsi ini mensimulasikan pergerakan aktor ke empat arah mata angin. Sesuai mekanisme es licin, aktor akan terus meluncur melewati petak es biasa (*), angka (0-9), dan petak keluar (O) hingga menabrak rintangan atau batu (X).

Langkah-langkah Pembangkitan:

1. Menentukan arah luncuran (Atas, Bawah, Kiri, Kanan).
2. Melakukan perulangan posisi hingga menemukan rintangan atau batas papan.
3. Memvalidasi petak angka: jika angka yang dilewati sesuai urutan **NextDigit**, maka target digit diperbarui.
4. Menghentikan luncuran jika menabrak batu dan mengembalikan posisi tepat sebelum batu tersebut.
5. Mengembalikan status gagal jika luncuran melewati lava (L) atau keluar dari batas papan tanpa hambatan.

Pseudocode *Successor Generation*:

```
procedure GenerateSuccessors(input board, currentState) →  
    himpunan successor states  
  
{ Mengembalikan daftar state successor yang valid dari state saat ini }  
  
Deklarasi  
    successors : list of State
```

```

directions : list of Direction ← [Up, Down, Left, Right]
nextState  : State
valid      : boolean
dir        : Direction

```

Algoritma :

```

successors ← []
for dir in directions do
    (nextState, valid) ← slide(board, currentState, dir)
    if valid then
        successors ← successors + [nextState]
    endif
endfor
return successors

```

procedure slide(input board, s : State, dir : Direction) →
 (State, cost : integer, valid : boolean)

{ Menggeser state s ke arah dir pada board, mengembalikan state baru,
 total cost, dan apakah gerakan tersebut valid }

Deklarasi

```

dx, dy      : integer
x, y        : integer
nx, ny      : integer
next_digit  : integer
total_cost  : integer
cell        : Cell

```

Algoritma :

```

(dx, dy) ← getDelta(dir)
(x, y)   ← (s.X, s.Y)
next_digit ← s.NextDigit
total_cost ← 0

while true do
    nx ← x + dx
    ny ← y + dy

    if isOutOfBounds(nx, ny) then
        return (null, 0, false)
    endif

    cell ← board.at(nx, ny)

    if cell.type = Rock then
        if (x = s.X) AND (y = s.Y) then
            return (null, 0, false)
        endif
    endif
endwhile

```

```

        endif
        return (State(x, y, next_digit), total_cost, true)
    endif

    if cell.type = Lava then
        return (null, 0, false)
    endif

    if cell.type = Number then
        if cell.digit = next_digit then
            next_digit ← next_digit + 1
        else if cell.digit > next_digit then
            return (null, 0, false)
        endif
    endif

    total_cost ← total_cost + cell.cost
    x ← nx
    y ← ny
endwhile

```

c. Implementasi *Unified Best-First Search* (UCS, GBFS, A*)

1. Konsep Dasar Pencarian

Dalam menyelesaikan persoalan pencarian rute pada Ice Sliding Puzzle, program ini merancang sebuah arsitektur pencarian yang disebut Unified Best-First Search. Alih-alih mengimplementasikan tiga fungsi pencarian yang terpisah dan redundan, Uniform Cost Search (UCS), Greedy Best-First Search (GBFS), dan A-Star (A*) disatukan ke dalam satu mesin inti yang beroperasi menggunakan struktur data *Priority Queue*. Pendekatan ini sangat efisien karena ketiga algoritma tersebut pada dasarnya memiliki alur penelusuran graf yang identik. Perbedaan utama di antara ketiganya murni terletak pada rumus penentuan nilai prioritas saat simpul dimasukkan ke dalam antrian. Program ini memasukkan fungsi evaluasi $f(n)$ secara dinamis saat inisialisasi pencarian dengan aturan matematis sebagai berikut:

- **UCS:** Memprioritaskan biaya aktual terkecil, $f(n) = g(n)$.
- **GBFS:** Memprioritaskan estimasi jarak terdekat, $f(n) = h(n)$.
- **A*:** Memprioritaskan gabungan biaya dan jarak, $f(n) = g(n) + h(n)$

2. Mekanisme Pemangkasan Graf

Untuk mencegah karakter terjebak dalam siklus pergerakan bolak-balik tanpa akhir (seperti meluncur berulang kali di antara dua batu), algoritma ini mengadopsi mekanisme *Graph Search*. Setiap kali sebuah simpul diekspansi, program akan mencatat biaya $g(n)$ terbaik untuk status tersebut di dalam memori pemetaan (*Hash Map*). Apabila pada iterasi selanjutnya algoritma menemukan rute lain yang menuju ke status yang sama persis namun dengan akumulasi biaya yang lebih mahal atau setara, cabang eksplorasi tersebut akan langsung dipangkas (*pruned*). Hal ini memastikan memori komputasi tidak terbuang sia-sia untuk mengevaluasi jalur suboptimal.

3. Langkah-Langkah Penelusuran Algoritma

Secara naratif, proses *Unified Best-First Search* berjalan melalui siklus berikut:

1. Program membentuk simpul akar (*root node*) berdasarkan posisi mula-mula **Z** dengan biaya $g(n) = 0$. Simpul ini dimasukkan ke dalam Antrean Prioritas.
2. Selama antrean belum kosong, program akan mengekstrak simpul teratas (simpul dengan nilai $f(n)$ paling kecil sesuai aturan algoritma yang dipilih).
3. Program memeriksa catatan memori. Jika simpul yang baru saja diekstrak ternyata memiliki biaya $g(n)$ yang lebih besar daripada biaya terbaik yang pernah dicatat untuk posisi tersebut, simpul ini diabaikan.
4. Program melakukan pengujian kondisi menang (*goal test*). Jika status karakter pada simpul ini berada di atas petak keluar (**O**) dan target angkanya telah terpenuhi secara utuh, penelusuran dihentikan dan lintasan langkah dikembalikan sebagai solusi.
5. Jika belum menang, program memanggil fungsi generator untuk membangkitkan seluruh probabilitas luncuran (suksesor) yang valid dari titik tersebut.
6. Untuk setiap suksesor yang dihasilkan, program menghitung biaya baru $g(n)$ serta estimasi jarak $h(n)$. Apabila suksesor ini membawa karakter ke status yang belum pernah dikunjungi, atau membawa karakter ke status lama namun melalui rute yang lebih murah, maka catatan memori akan diperbarui dan suksesor dimasukkan ke dalam antrean. Langkah dieksekusi berulang kembali ke poin 2.

4. Pseudocode *Unified Best-First Search*

```
procedure UnifiedSearch(input config : SearchConfig) →  
    hasil pencarian (SearchResult) atau null jika tidak ditemukan  
  
{ Mengembalikan hasil pencarian berupa path solusi menggunakan  
algoritma  
    yang ditentukan oleh config. Berlaku untuk UCS, GBFS, maupun A*  
    tergantung fungsi prioritas yang digunakan }  
  
Deklarasi  
    pq          : priority queue of Node  
    best_g      : map of (Key → integer)  
    root        : Node  
    root_h      : integer  
    node        : Node  
    child_g     : integer  
    child_h     : integer  
    child_node  : Node  
    succ        : Successor  
  
Algoritma :  
    { pq diurutkan secara otomatis berdasarkan aturan f(n)  
    algoritma yang dipilih }  
    pq ← newPriorityQueue(priorityFor(config.Algorithm))  
  
    { best_g bertindak sebagai closed list untuk mencegah siklus  
    dan redundansi }  
    best_g ← {}  
  
    { Inisialisasi simpul akar }  
    root_h ← config.Heuristic(config.Start, config.Board)  
    root ← Node(state : config.Start,  
                g      : 0,  
                h      : root_h)  
  
    pq.push(root)  
    best_g[config.Start.Key()] ← 0  
  
    while not pq.isEmpty() do  
        node ← pq.pop()  
  
        { Pangkas simpul jika ditemukan jalur lebih murah di  
        iterasi sebelumnya }  
        if (best_g contains node.state.Key()) AND  
            (best_g[node.state.Key()] < node.g) then  
            continue  
        endif
```

```

        { Cek kondisi kemenangan }
        if isGoal(config.Board, node) then
            return buildResult(node)
        endif

        { Bangkitkan seluruh kemungkinan pergerakan (sukesor) }
        for succ in config.Generator.Generate(config.Board,
node.state) do
            child_g ← node.g + succ.stepCost

            { Masukkan ke antrean jika rute ini adalah rute
optimal terbaru }
            if (NOT best_g contains succ.state.Key()) OR
                (child_g < best_g[succ.state.Key()]) then

                best_g[succ.state.Key()] ← child_g
                child_h ← config.Heuristic(succ.state,
config.Board)

                child_node ← Node(state : succ.state,
                                parent : node,
                                g      : child_g,
                                h      : child_h)

                pq.push(child_node)
            endif
        endfor
    endwhile

    return null { solusi gagal ditemukan }

```

d. Algoritma Alternatif: Iterative Deepening A* (IDA*) (Bonus)

1. Konsep Dasar IDA*

IDA* dikembangkan untuk menutupi kelemahan fatal dari algoritma A* standar, yaitu kompleksitas ruangnya (*Space Complexity*) yang sangat boros memori.

Alih-alih menyimpan seluruh simpul hidup di dalam antrean prioritas yang memakan banyak *Random Access Memory* (RAM), IDA* menggunakan pendekatan pencarian mendalam atau *Depth-First Search* (DFS). Pencarian ini dipandu oleh fungsi evaluasi $f(n) = g(n) + h(n)$ dan dikendalikan oleh sebuah ambang batas (*threshold / bound*).

2. Mekanisme *Threshold Pruning*

Algoritma akan mengeksplorasi suatu rute secara mendalam (rekursif) selagi nilai $f(n)$ dari simpul di rute tersebut masih di bawah atau sama dengan nilai *threshold*. Jika saat menelusuri suatu cabang algoritma menemukan simpul dengan $f(n) > threshold$, maka cabang tersebut akan langsung dipangkas (*pruned*) dan penelusuran mundur (*backtrack*) ke cabang lain.

Jika seluruh kemungkinan cabang telah dicoba namun solusi belum ditemukan, algoritma akan memperbarui nilai batas. Nilai batas yang baru adalah nilai minimum dari semua nilai $f(n)$ yang sebelumnya terpotong (melebihi batas lama). Setelah batas dinaikkan, iterasi pencarian diulang kembali dari simpul awal.

3. Pseudocode IDA*

```
procedure DFS(input node      : Node,
              input board     : Board,
              input current_bound: integer,
              input path_visited : Set of State) →
    (found : boolean, new_bound : integer, goal_node : Node)

{ Penelusuran DFS rekursif untuk IDA*. Mengembalikan apakah goal
  ditemukan,
  batas f minimum yang melampaui current_bound, dan simpul goal
  jika ditemukan }

Deklarasi
    f           : integer
    min_outlier : integer
    child_g     : integer
    child_h     : integer
    child       : Node
    found       : boolean
    new_bound   : integer
    goal_node   : Node
    succ        : Successor

Algoritma :
    f ← node.g + node.h

    { Pangkas jika f melebihi batas iterasi saat ini }
    if f > current_bound then
        return (false, f, null)
    endif
```

```

    { Cek kondisi kemenangan }
    if isGoal(board, node) then
        return (true, f, node)
    endif

    min_outlier ← ∞

    { Tandai state saat ini sebagai sedang dikunjungi di jalur
    ini }
    path_visited.add(node.state)

    for succ in GenerateSuccessors(board, node.state) do
        { Mencegah karakter meluncur bolak-balik di rute yang
    sama }
        if succ.state in path_visited then
            continue
        endif

        child_g ← node.g + succ.stepCost
        child_h ← calculateHeuristic(succ.state, board)
        child ← Node(state : succ.state,
                        parent : node,
                        g      : child_g,
                        h      : child_h)

        { Telusuri lebih dalam secara rekursif }
        (found, new_bound, goal_node) ← DFS(child, board,
        current_bound, path_visited)

        if found then
            return (true, new_bound, goal_node)
        endif

        if new_bound < min_outlier then
            min_outlier ← new_bound
        endif
    endfor

    { Backtrack: hapus state dari jalur yang sedang dikunjungi }
    path_visited.remove(node.state)

    return (false, min_outlier, null)

procedure IDAStar(input start_state : State,
                  input board       : Board) →
    path solusi atau null jika tidak ditemukan

```

```
{ Mengembalikan path solusi menggunakan algoritma IDA* dengan  
  iterative deepening berbasis nilai  $f(n) = g(n) + h(n)$  }
```

Deklarasi

```
start_h      : integer  
root         : Node  
bound        : integer  
found        : boolean  
new_bound    : integer  
goal_node    : Node  
path_visited: Set of State
```

Algoritma :

```
start_h      ← calculateHeuristic(start_state, board)  
root         ← Node(state : start_state,  
                    g      : 0,  
                    h      : start_h)
```

```
{ Batas awal adalah estimasi heuristik dari titik mulai }
```

```
bound        ← root.h
```

```
path_visited ← {}
```

```
{ Loop utama Iterative Deepening }
```

```
while true do
```

```
  (found, new_bound, goal_node) ← DFS(root, board, bound,  
  path_visited)
```

```
  if found then
```

```
    return buildPath(goal_node)
```

```
  endif
```

```
  if new_bound =  $\infty$  then
```

```
    return null { tidak ada rute penyelesaian sama sekali
```

```
  }
```

```
  endif
```

```
  { Naikkan batas untuk iterasi pencarian selanjutnya }
```

```
  bound ← new_bound
```

```
endwhile
```

e. Implementasi Fungsi Heuristik

Bagian ini menguraikan implementasi dan integrasi fungsi heuristik $h(n)$ dalam algoritma pathfinding untuk penyelesaian Ice Sliding Puzzle. Pada Tugas Kecil 3 ini,

diimplementasikan 5 fungsi heuristik (termasuk heuristik bonus) yang bertugas untuk memperkirakan biaya perjalanan dari simpul saat ini ke simpul tujuan.

Mengingat aturan permainan mengharuskan aktor melewati angka secara berurutan, seluruh heuristik ini dirancang untuk berfokus pada target terdekat yang relevan secara urutan (angka berikutnya), dan baru akan menargetkan pintu keluar (*exit goal*) setelah seluruh angka terkumpul. Berikut adalah penjelasan untuk masing-masing heuristik:

1. Heuristik Zero

Heuristik Zero merupakan instrumen heuristik dasar yang secara konstan memberikan estimasi biaya nol untuk setiap keadaan pada ruang status. Dalam konteks pemecahan masalah pencarian, heuristik ini secara khusus diintegrasikan pada algoritma Uniform Cost Search (UCS). Dengan menetapkan nilai estimasi selalu nol, fungsi prioritas pencarian menjadi murni bergantung pada biaya aktual sejauh ini, yakni $f(n) = g(n)$. Hal ini memastikan Uniform Cost Search berperilaku mengekskansi simpul murni berdasarkan biaya terendah sejauh ini yang menjamin tercapainya solusi optimal tanpa memerlukan panduan heuristik.

2. Manhattan Distance

Jarak Manhattan merupakan metrik spasial yang menghitung jumlah total panjang segmen garis antara dua titik diukur terhadap setiap sumbu dalam sistem koordinat Kartesius. Secara matematis, untuk permasalahan Ice Sliding Puzzle, jarak Manhattan diformulasikan sebagai jumlah selisih absolut antara posisi aktor saat ini dengan sel target (angka target berikutnya atau pintu keluar) pada baris dan kolom yang bersesuaian.

$$h(n) = |x_{target} - x_{aktor}| + |y_{target} - y_{aktor}|$$

Implementasi fungsi heuristik $h(n)$ berbasis Manhattan pada Ice Sliding Puzzle menghitung perkiraan langkah minimal secara horizontal atau vertikal dari posisi aktor menuju target. Heuristik ini dijamin admissible (tidak pernah melebihi-lebihkan biaya sebenarnya) karena pergerakan aktor hanya dapat terjadi dalam orientasi horizontal dan vertikal (sliding). Dengan asumsi setiap petak es memiliki biaya minimal 1, estimasi ini menjadi batas bawah nyata dari total biaya

yang diperlukan, sehingga sangat relevan dan aman untuk menjamin optimalitas pada algoritma A* Search dan IDA* Search.

3. Remaining Digits

Remaining Digits adalah sebuah heuristik yang memperkirakan biaya berdasarkan jumlah digit (angka) yang masih harus dikumpulkan secara berurutan oleh aktor sebelum jalur menuju pintu keluar terbuka. Secara matematis, heuristik ini diformulasikan dengan mengalikan jumlah digit yang tersisa dengan biaya minimum dari petak yang dapat dilintasi (*minimum traversable cost*) di atas konfigurasi papan permainan saat itu.

$$h(n) = \text{Remaining Digits} \times \text{minCost}$$

Dalam penyelesaian Ice Sliding Puzzle, heuristik ini sangat berguna karena mencerminkan tingkat sisa tahapan yang wajib diselesaikan. Heuristik ini bersifat admissible karena untuk mengumpulkan setiap digit angka yang tersisa, aktor setidaknya harus melakukan satu kali pergerakan melintasi petak dengan biaya paling minimal. Hal ini memberikan estimasi awal tentang seberapa banyak langkah perpindahan yang diperlukan sebelum tujuan akhir benar-benar dapat dicapai.

4. Euclidean Distance (Bonus)

Jarak Euclidean merupakan metrik spasial yang menghitung panjang garis lurus terpendek antara dua titik dalam ruang Euclidean dua dimensi. Secara matematis, heuristik ini diformulasikan sebagai akar kuadrat dari jumlah kuadrat selisih koordinat posisi aktor saat ini dengan sel target, yang kemudian dibulatkan ke bawah.

$$h(n) = \sqrt{(x_{\text{target}} - x_{\text{aktor}})^2 + (y_{\text{target}} - y_{\text{aktor}})^2}$$

Dalam pemecahan masalah pencarian dalam ruang grid, jarak Euclidean memberikan estimasi yang lebih konservatif jika dibandingkan dengan jarak Manhattan. Karena perhitungan garis lurus selalu menghasilkan nilai yang lebih kecil atau sama dengan perhitungan jarak Manhattan, heuristik Euclidean ini tetap

terjamin admissible. Meskipun estimasi yang lebih kecil berpotensi menyebabkan algoritma mengekskansi simpul sedikit lebih banyak dibandingkan Manhattan, heuristik ini tetap menjadi instrumen valid untuk memandu pencarian menuju jalur keluar optimal.

5. Max Combined (Bonus)

Max Combined adalah sebuah heuristik tingkat lanjut yang mengintegrasikan dua pendekatan heuristik yang berbeda, yakni Manhattan Distance dan Remaining Digits, dengan mengevaluasi dan mengambil nilai estimasi maksimum di antara keduanya pada setiap keadaan simpul.

Secara matematis, pendekatan ini diformulasikan sebagai:

$$h(n) = \max(h_{\text{manhattan}}(n), h_{\text{Remaining Digits}}(n))$$

Penggunaan heuristik gabungan ini menjadi instrumen yang sangat penting karena sifatnya yang lebih dominan. Jika terdapat dua heuristik yang secara teoritis terbukti admissible, mengambil batasan maksimum dari keduanya akan menghasilkan nilai estimasi baru yang lebih tajam dan paling mendekati biaya aktual sebenarnya, tanpa melanggar syarat admissibility. Dengan demikian, Max Combined bertindak sebagai indikator dominan dalam memilih simpul yang paling menjanjikan untuk diekspansi, sehingga secara signifikan memangkas ruang pencarian dan mempercepat proses menemukan jalur keluar optimal pada algoritma A* Search dan IDA* Search.

C. Analisis Strategi Algoritma

a. Definisi Fungsi Evaluasi $f(n)$ dan $g(n)$

Dalam implementasi algoritma pencarian yang digunakan (*Uniform Cost Search*, *Greedy Best-First Search*, *A-Star*, dan *Iterative Deepening A**), penentuan prioritas langkah dievaluasi berdasarkan fungsi matematis. Variabel di dalam fungsi tersebut didefinisikan sebagai berikut:

- Fungsi $g(n)$ (Biaya Aktual): Merupakan akumulasi *cost* pasti yang telah dikeluarkan karakter dari status awal untuk mencapai status n . Sesuai aturan permainan, $g(n)$

dihitung dari penjumlahan nilai pada setiap petak yang dilewati selama karakter meluncur, tidak termasuk petak tempat ia mulai meluncur, namun termasuk petak tempat ia berhenti.

- Fungsi $h(n)$ (Heuristik): Merupakan estimasi atau tebakan biaya terendah yang dibutuhkan karakter dari status n saat ini untuk mencapai tujuan akhir (petak angka selanjutnya atau petak keluar 'O').
- Fungsi $f(n)$ (Fungsi Evaluasi): Merupakan rumusan prioritas utama algoritma.
 - Pada UCS, algoritma hanya mementingkan biaya aktual, sehingga $f(n) = g(n)$.
 - Pada GBFS, algoritma secara *greedy* hanya mementingkan estimasi jarak terdekat, sehingga $f(n) = h(n)$.
 - Pada A* dan IDA*, algoritma menyeimbangkan biaya yang telah dikeluarkan dan sisa jarak ke tujuan, sehingga $f(n) = g(n) + h(n)$

b. Pembuktian Admissibility Heuristik A*

Bagian ini memaparkan validasi matematis terhadap sifat admissibility dari fungsi-fungsi heuristik yang diimplementasikan. Sebuah fungsi heuristik $h(n)$ diklasifikasikan sebagai admissible (dapat diterima) jika untuk setiap simpul n memenuhi kriteria:

$$h(n) \leq h^*(n)$$

Di mana $h^*(n)$ merepresentasikan biaya optimal sesungguhnya dari keadaan simpul n menuju tujuan akhir (*goal state*). Sifat ini bersifat krusial karena menjamin algoritma A* Search akan selalu menemukan solusi paling optimal.

1. Dasar Formal dan Asumsi

Analisis pembuktian ini didasarkan pada mekanisme pergerakan sliding di mana setiap sel yang dilintasi aktor dalam satu rangkaian gerak memberikan kontribusi terhadap biaya akumulatif. Pembuktian ini berpijak pada asumsi bahwa seluruh petak yang dapat dilalui (bukan batu atau lava) memiliki biaya minimum $c_{min} \geq 1$. Asumsi ini menjadi landasan karena biaya nol akan meniadakan konsep urgensi biaya dalam sistem puzzle.

Berdasarkan sifat norma L1 pada grid dua dimensi, jarak Manhattan merupakan batas bawah jumlah sel minimal yang harus dilewati dalam rangkaian gerakan aksial. Oleh karena setiap sel memiliki biaya minimal satu, maka biaya nyata untuk berpindah antar titik dipastikan akan selalu lebih besar atau sama dengan jumlah langkah selisih koordinatnya.

2. Analisis Admissibility per Fungsi Heuristik

- **Manhattan Distance**

Fungsi ini terbukti admissible karena pergerakan aktor hanya terbatas pada sumbu horizontal dan vertikal. Mengingat biaya minimum tiap petak adalah satu, maka biaya optimal sesungguhnya (h^*) untuk mencapai target tersebut dipastikan akan selalu lebih besar atau sama dengan nilai jarak Manhattan. Dengan demikian, heuristik ini tidak pernah melebihi-lebihkan biaya nyata yang diperlukan untuk mencapai target angka berikutnya maupun pintu keluar.

- **Remaining Digits**

Heuristik ini memberikan estimasi berdasarkan jumlah angka (digit) yang masih harus dikumpulkan secara berurutan. Mengingat setiap angka berada pada lokasi sel yang berbeda dan wajib dikunjungi satu per satu sesuai urutan, maka aktor setidaknya harus melintasi sel-sel tersebut yang masing-masing memiliki biaya minimal tertentu. Karena akumulasi biaya nyata mencakup seluruh rangkaian kunjungan sel angka ini, maka nilai estimasi Remaining Digits secara matematis akan selalu berada di bawah atau sama dengan biaya optimal sesungguhnya.

- **Euclidean Distance (Bonus)**

Jarak Euclidean menghitung panjang garis lurus antara aktor dan target. Secara geometris dalam ruang grid, nilai jarak Euclidean akan selalu lebih kecil atau sama dengan jarak Manhattan. Karena jarak Manhattan telah terbukti admissible dalam menjamin optimalitas, maka jarak Euclidean yang memberikan estimasi lebih konservatif secara otomatis memenuhi syarat admissibility.

- **Max Combined (Bonus)**

Berdasarkan teorema umum dalam strategi algoritma, jika terdapat dua fungsi heuristik yang keduanya bersifat *admissible*, maka nilai maksimal di antara keduanya juga tetap bersifat *admissible*. Dengan mengambil batasan tertinggi dari estimasi Manhattan dan Remaining Digits, dihasilkan nilai baru yang lebih dominan dan mendekati biaya aktual tanpa melanggar prinsip dasar penjaminan solusi optimal. Hal ini memungkinkan algoritma untuk bekerja lebih terarah dan efisien.

c. Perbandingan Teoritis

Berikut adalah komparasi teoritis dari keempat algoritma yang diimplementasikan berdasarkan parameter kelengkapan (*completeness*), optimalitas, serta kompleksitas waktu dan ruang:

1. Uniform Cost Search (UCS)

- **Kelebihan:** Bersifat komplit dan dijamin optimal. Algoritma ini dipastikan akan menemukan rute dengan biaya paling murah jika solusi memang eksis.
- **Kekurangan:** Penelusuran buta (*uninformed search*). UCS akan mengekskansi simpul ke segala arah secara radial karena tidak memiliki arah tujuan. Kompleksitas ruang dan waktunya sangat tinggi akibat besarnya jumlah *node* tidak relevan yang harus disimpan di memori antrean prioritas.

2. Greedy Best-First Search (GBFS)

- **Kelebihan:** Pencarian terpandu yang sangat cepat. GBFS fokus murni pada pendekatan ke target secara spasial, membuat jumlah iterasi dan ekspansi simpulnya sangat minim.
- **Kekurangan:** Tidak optimal dan berpotensi tidak komplit. GBFS sangat rentan terjebak pada jalan buntu (optimum lokal) atau menemukan rute yang sangat mahal karena sama sekali mengabaikan akumulasi *cost* di belakangnya.

3. A-Star (A*)

- **Kelebihan:** Menggabungkan keandalan optimalitas milik UCS dan efisiensi penelusuran terarah milik GBFS. Apabila menggunakan heuristik yang *admissible*, A* akan menghasilkan solusi paling efisien dan meminimalkan ekspansi simpul berlebih.

- **Kekurangan:** Keterbatasan terbesar A* berada pada kompleksitas ruangnya. Karena harus melacak batas eksplorasi, A* menyimpan seluruh simpul di dalam memori (*Priority Queue*). Pada peta puzzle dengan ukuran masif, algoritma ini sangat rakus memori dan dapat mengakibatkan *Out of Memory*.

4. Iterative Deepening A-Star (IDA*)

- **Kelebihan:** Sebagai algoritma bonus alternatif, IDA* hadir untuk menyelesaikan kelemahan utama A*. IDA* mengeksplorasi pohon pergerakan menggunakan DFS, sehingga kompleksitas ruangnya turun secara drastis menjadi linear ($O(bd)$). IDA* tetap memberikan jaminan komplit dan rute yang optimal.
- **Kekurangan:** Karena *threshold* selalu dinaikkan bertahap, IDA* akan membuat waktu (*overhead*) dengan terus menerus mengekspansi ulang simpul-simpul di lapis atas pada setiap iterasi yang baru.

D. Source Code

Komponen	Kode
main.go	<pre> package main import ("bufio" "flag" "fmt" "os" "strings" "iceslide/src/board" "iceslide/src/gui" "iceslide/src/heuristic" "iceslide/src/search" "iceslide/src/state" "iceslide/src/successor" "iceslide/src/visualizer") </pre>

```

const defaultSavePath = "test/solusi.txt"

func main() {
    if err := run(); err != nil {
        fmt.Fprintln(os.Stderr, "error:", err)
        os.Exit(1)
    }
}

func run() error {
    useGUI := flag.Bool("gui", false, "jalankan mode GUI")
    algName := flag.String("alg", "", "algoritma: ucs | gbfs |
    astar | idastar")
    hName := flag.String("h", "", "heuristik: zero | manhattan |
    remaining | euclidean | maxcombined")
    outputPath := flag.String("out", defaultSavePath, "default file
    output untuk fitur simpan solusi")
    flag.Parse()

    if *useGUI || flag.NArg() == 0 {
        gui.Run()
        return nil
    }

    args := flag.Args()
    pr, err := board.LoadFile(args[0])
    if err != nil {
        return fmt.Errorf("load %s: %w", args[0], err)
    }
    start := state.State{X: pr.StartX, Y: pr.StartY, NextDigit: 0}

    scanner := bufio.NewScanner(os.Stdin)

    alg, err := chooseAlg(scanner, os.Stdout, *algName)
    if err != nil {
        return err
    }
    h, hLabel, err := chooseHeuristic(scanner, os.Stdout, alg,

```

```

*hName)
    if err != nil {
        return err
    }

    res, err := search.Search(search.Config{
        Board:    pr.Board,
        Start:    start,
        Algorithm: alg,
        Heuristic: h,
        Generator: successor.SlideGenerator{},
    })
    if err != nil {
        return err
    }

    visualizer.PrintSummary(os.Stdout, alg, hLabel, pr.Board,
start, res)

    return visualizer.RunInteractive(
        scanner, os.Stdout,
        alg, hLabel,
        pr.Board, start, res,
        *outPath,
    )
}

func chooseAlg(scanner *bufio.Scanner, w *os.File, flagVal string)
(search.Algorithm, error) {
    if flagVal != "" {
        return parseAlg(flagVal)
    }
    return visualizer.SelectAlgorithm(scanner, w)
}

func chooseHeuristic(scanner *bufio.Scanner, w *os.File, alg
search.Algorithm, flagVal string) (heuristic.Func, string, error) {
    if alg == search.UCS {

```

```

        return heuristic.Zero, "Zero (UCS)", nil
    }
    if flagVal != "" {
        return parseHeuristic(flagVal)
    }
    return visualizer.SelectHeuristic(scanner, w, alg)
}

func parseAlg(name string) (search.Algorithm, error) {
    switch strings.ToLower(name) {
    case "ucs":
        return search.UCS, nil
    case "gbfs":
        return search.GBFS, nil
    case "astar", "a*":
        return search.AStar, nil
    case "idastar", "ida*":
        return search.IDAStar, nil
    default:
        return 0, fmt.Errorf("algoritma tidak dikenal: %q", name)
    }
}

func parseHeuristic(name string) (heuristic.Func, string, error) {
    switch strings.ToLower(name) {
    case "zero":
        return heuristic.Zero, "Zero", nil
    case "manhattan":
        return heuristic.Manhattan, "Manhattan", nil
    case "remaining":
        return heuristic.RemainingDigits, "RemainingDigits", nil
    case "euclidean":
        return heuristic.Euclidean, "Euclidean", nil
    case "maxcombined":
        return heuristic.MaxCombined, "Max Combined", nil
    default:
        return nil, "", fmt.Errorf("heuristik tidak dikenal: %q",
name)
    }
}

```

	<pre> } } </pre>
cell.go	<pre> package board type CellType uint8 const (CellIce CellType = iota CellRock CellLava CellExit CellNumber) type Cell struct { Type CellType Cost int Digit int } </pre>
board.go	<pre> package board type Board struct { width int height int grid [][]Cell maxDigit int } func New(grid [][]Cell) *Board { height := len(grid) width := 0 if height > 0 { width = len(grid[0]) } maxDigit := -1 for _, row := range grid { </pre>

	<pre> for _, c := range row { if c.Type == CellNumber && c.Digit > maxDigit { maxDigit = c.Digit } } } return &Board{ width: width, height: height, grid: grid, maxDigit: maxDigit, } } func (b *Board) Width() int { return b.width } func (b *Board) Height() int { return b.height } func (b *Board) MaxDigit() int { return b.maxDigit } func (b *Board) At(x, y int) Cell { return b.grid[y][x] } func (b *Board) InBounds(x, y int) bool { return x >= 0 && x < b.width && y >= 0 && y < b.height } </pre>
parser.go	<pre> package board import ("bufio" "errors" "fmt" "io" "os" "strconv" "strings") </pre>

```

type ParseResult struct {
    Board *Board
    StartX int
    StartY int
}

func LoadFile(path string) (*ParseResult, error) {
    f, err := os.Open(path)
    if err != nil {
        return nil, fmt.Errorf("open %s: %w", path, err)
    }
    defer f.Close()
    return Parse(f)
}

func Parse(r io.Reader) (*ParseResult, error) {
    scanner := bufio.NewScanner(r)
    scanner.Buffer(make([]byte, 0, 64*1024), 1<<20)

    N, M, err := parseDims(scanner)
    if err != nil {
        return nil, err
    }

    grid, startX, startY, err := parseGrid(scanner, N, M)
    if err != nil {
        return nil, err
    }

    if err := parseCosts(scanner, grid, N, M); err != nil {
        return nil, err
    }

    if err := scanner.Err(); err != nil {
        return nil, fmt.Errorf("read: %w", err)
    }

    b := New(grid)

```

```

    if err := validateBoard(b); err != nil {
        return nil, err
    }

    return &ParseResult{
        Board:  b,
        StartX: startX,
        StartY: startY,
    }, nil
}

func validateBoard(b *Board) error {
    hasExit := false
    for y := 0; y < b.Height(); y++ {
        for x := 0; x < b.Width(); x++ {
            if b.At(x, y).Type == CellExit {
                hasExit = true
            }
        }
    }
    if !hasExit {
        return errors.New("invalid board: no exit tile (0) found")
    }

    for d := 0; d <= b.MaxDigit(); d++ {
        found := false
        for y := 0; y < b.Height() && !found; y++ {
            for x := 0; x < b.Width() && !found; x++ {
                c := b.At(x, y)
                if c.Type == CellNumber && c.Digit == d {
                    found = true
                }
            }
        }
        if !found {
            return fmt.Errorf("invalid board: digit %d missing (gap in sequence 0..%d)", d, b.MaxDigit())
        }
    }
}

```

```

    }
}
return nil
}

func parseDims(s *bufio.Scanner) (int, int, error) {
    if !s.Scan() {
        if err := s.Err(); err != nil {
            return 0, 0, fmt.Errorf("read dims: %w", err)
        }
        return 0, 0, errors.New("empty input")
    }
    fields := strings.Fields(s.Text())
    if len(fields) != 2 {
        return 0, 0, fmt.Errorf("dims line: want \"N M\", got %q",
s.Text())
    }
    N, err := strconv.Atoi(fields[0])
    if err != nil {
        return 0, 0, fmt.Errorf("dims N: %w", err)
    }
    M, err := strconv.Atoi(fields[1])
    if err != nil {
        return 0, 0, fmt.Errorf("dims M: %w", err)
    }
    if N <= 0 || M <= 0 {
        return 0, 0, fmt.Errorf("dims must be positive: N=%d M=%d",
N, M)
    }
    return N, M, nil
}

func parseGrid(s *bufio.Scanner, N, M int) ([][]Cell, int, int,
error) {
    grid := make([][]Cell, N)
    startX, startY := -1, -1
    for y := 0; y < N; y++ {
        if !s.Scan() {

```

```

        return nil, 0, 0, fmt.Errorf("expected %d board rows,
got %d", N, y)
    }
    runes := []rune(s.Text())
    if len(runes) < M {
        return nil, 0, 0, fmt.Errorf("board row %d: want %d
cols, got %d", y, M, len(runes))
    }
    row := make([]Cell, M)
    for x := 0; x < M; x++ {
        cell, isStart, err := glyphToCell(runes[x])
        if err != nil {
            return nil, 0, 0, fmt.Errorf("board row %d col %d:
%w", y, x, err)
        }
        if isStart {
            if startX != -1 {
                return nil, 0, 0, fmt.Errorf("multiple start
tiles: (%d,%d) and (%d,%d)", startX, startY, x, y)
            }
            startX, startY = x, y
        }
        row[x] = cell
    }
    grid[y] = row
}
if startX == -1 {
    return nil, 0, 0, errors.New("no start tile (Z) found")
}
return grid, startX, startY, nil
}

func parseCosts(s *bufio.Scanner, grid [][]Cell, N, M int) error {
    for y := 0; y < N; y++ {
        if !s.Scan() {
            return fmt.Errorf("expected %d cost rows, got %d", N,
y)
        }
    }
}

```

```

        fields := strings.Fields(s.Text())
        if len(fields) < M {
            return fmt.Errorf("cost row %d: want %d values, got %d", y, M, len(fields))
        }
        for x := 0; x < M; x++ {
            cost, err := strconv.Atoi(fields[x])
            if err != nil {
                return fmt.Errorf("cost row %d col %d: %w", y, x, err)
            }
            grid[y][x].Cost = cost
        }
    }
    return nil
}

func glyphToCell(ch rune) (Cell, bool, error) {
    switch ch {
    case 'X':
        return Cell{Type: CellRock}, false, nil
    case '*':
        return Cell{Type: CellIce}, false, nil
    case 'L':
        return Cell{Type: CellLava}, false, nil
    case 'O':
        return Cell{Type: CellExit}, false, nil
    case 'Z':
        return Cell{Type: CellIce}, true, nil
    case '0', '1', '2', '3', '4', '5', '6', '7', '8', '9':
        return Cell{Type: CellNumber, Digit: int(ch - '0')}, false, nil
    default:
        return Cell{}, false, fmt.Errorf("unknown glyph %q", ch)
    }
}

```

state.go

package state

```

type State struct {
    X int
    Y int
    NextDigit int
}

func (s State) Key() uint64 {
    const m24 = uint64(1)<<24 - 1
    const m16 = uint64(1)<<16 - 1
    return (uint64(s.NextDigit)&m16)<<48 |
        (uint64(s.Y)&m24)<<24 |
        (uint64(s.X) & m24)
}

func (s State) Equals(other State) bool {
    return s == other
}

```

successor.go

```

package successor

import (
    "iceslide/src/board"
    "iceslide/src/state"
)

type Direction uint8

const (
    Up Direction = iota
    Down
    Left
    Right
)

func (d Direction) String() string {
    switch d {
    case Up:

```

```

        return "U"
    case Down:
        return "D"
    case Left:
        return "L"
    case Right:
        return "R"
    default:
        return "?"
    }
}

type Successor struct {
    State state.State
    StepCost int
    Direction Direction
}

type Generator interface {
    Generate(b *board.Board, s state.State) []Successor
}

type SlideGenerator struct{}

var allDirections = [...]Direction{Up, Down, Left, Right}

func (g SlideGenerator) Generate(b *board.Board, s state.State)
[]Successor {
    out := make([]Successor, 0, len(allDirections))
    for _, d := range allDirections {
        if succ, ok := g.slide(b, s, d); ok {
            out = append(out, succ)
        }
    }
    return out
}

func (g SlideGenerator) slide(b *board.Board, s state.State, d

```

```

Direction) (Successor, bool) {
    dx, dy := delta(d)
    x, y := s.X, s.Y
    nextDigit := s.NextDigit
    cost := 0

    for {
        nx, ny := x+dx, y+dy

        if !b.InBounds(nx, ny) {
            return Successor{}, false
        }

        cell := b.At(nx, ny)

        switch cell.Type {
        case board.CellRock:
            if x == s.X && y == s.Y {
                return Successor{}, false
            }
            return Successor{
                State: state.State{X: x, Y: y, NextDigit:
nextDigit},
                StepCost: cost,
                Direction: d,
            }, true

        case board.CellLava:
            return Successor{}, false

        case board.CellNumber:
            if cell.Digit > nextDigit {
                return Successor{}, false
            }
            if cell.Digit == nextDigit {
                nextDigit++
            }
            cost += cell.Cost

```

```

        x, y = nx, ny

        case board.CellExit, board.CellIce:
            cost += cell.Cost
            x, y = nx, ny
        }
    }
}

func delta(d Direction) (int, int) {
    switch d {
    case Up:
        return 0, -1
    case Down:
        return 0, 1
    case Left:
        return -1, 0
    case Right:
        return 1, 0
    default:
        return 0, 0
    }
}

```

heuristic.go

```

package heuristic

import (
    "math"

    "iceslide/src/board"
    "iceslide/src/state"
)

type Func func(s state.State, b *board.Board) int

func Zero(s state.State, b *board.Board) int { return 0 }

func Manhattan(s state.State, b *board.Board) int {

```

```

tx, ty, ok := findTarget(s, b)
if !ok {
    return 0
}
return abs(tx-s.X) + abs(ty-s.Y)
}

func RemainingDigits(s state.State, b *board.Board) int {
    remaining := b.MaxDigit() + 1 - s.NextDigit
    if remaining <= 0 {
        return 0
    }
    minCost := minTraversableCost(b)
    return remaining * minCost
}

func minTraversableCost(b *board.Board) int {
    min := math.MaxInt32
    for y := 0; y < b.Height(); y++ {
        for x := 0; x < b.Width(); x++ {
            c := b.At(x, y)
            if c.Type != board.CellRock && c.Type != board.CellLava
{
                if c.Cost < min {
                    min = c.Cost
                }
            }
        }
    }
    if min == math.MaxInt32 {
        return 0
    }
    return min
}

// bonus: Euclidean
func Euclidean(s state.State, b *board.Board) int {
    tx, ty, ok := findTarget(s, b)

```

```

    if !ok {
        return 0
    }
    dx := float64(tx - s.X)
    dy := float64(ty - s.Y)
    return int(math.Sqrt(dx*dx + dy*dy))
}

// bonus: MaxCombined
func MaxCombined(s state.State, b *board.Board) int {
    m := Manhattan(s, b)
    r := RemainingDigits(s, b)
    if m > r {
        return m
    }
    return r
}

func findTarget(s state.State, b *board.Board) (int, int, bool) {
    if s.NextDigit <= b.MaxDigit() {
        for y := 0; y < b.Height(); y++ {
            for x := 0; x < b.Width(); x++ {
                c := b.At(x, y)
                if c.Type == board.CellNumber && c.Digit ==
s.NextDigit {
                    return x, y, true
                }
            }
        }
    }
    for y := 0; y < b.Height(); y++ {
        for x := 0; x < b.Width(); x++ {
            if b.At(x, y).Type == board.CellExit {
                return x, y, true
            }
        }
    }
    return 0, 0, false
}

```

	<pre>} func abs(n int) int { if n < 0 { return -n } return n }</pre>
algorithm.go	<pre>package search type Algorithm uint8 const (UCS Algorithm = iota GBFS AStar IDAStar) func (a Algorithm) String() string { switch a { case UCS: return "UCS" case GBFS: return "GBFS" case AStar: return "A*" case IDAStar: return "IDA*" default: return "unknown" } } func priorityFor(a Algorithm) func(*Node) int { switch a { case UCS:</pre>

	<pre> return func(n *Node) int { return n.G } case GBFS: return func(n *Node) int { return n.H } case AStar: return func(n *Node) int { return n.G + n.H } default: return func(n *Node) int { return n.G + n.H } } } </pre>
node.go	<pre> package search import ("iceslide/src/state" "iceslide/src/successor") type Node struct { State state.State Parent *Node Action successor.Direction G int H int } func (n *Node) F() int { return n.G + n.H } func (n *Node) Path() []*Node { depth := 0 for cur := n; cur != nil; cur = cur.Parent { depth++ } path := make([]*Node, depth) for cur := n; cur != nil; cur = cur.Parent { depth-- path[depth] = cur } return path } </pre>

	<pre> }</pre>
priority_queue.go	<pre> package search type priorityQueue struct { items []*Node priority func(*Node) int } func newPriorityQueue(priority func(*Node) int) *priorityQueue { return &priorityQueue{priority: priority} } func (pq *priorityQueue) Len() int { return len(pq.items) } func (pq *priorityQueue) Less(i, j int) bool { return pq.priority(pq.items[i]) < pq.priority(pq.items[j]) } func (pq *priorityQueue) Swap(i, j int) { pq.items[i], pq.items[j] = pq.items[j], pq.items[i] } func (pq *priorityQueue) Push(x any) { pq.items = append(pq.items, x.(*Node)) } func (pq *priorityQueue) Pop() any { n := len(pq.items) item := pq.items[n-1] pq.items = pq.items[:n-1] return item }</pre>
search.go	<pre> package search</pre>

```
import (  
    "container/heap"  
    "errors"  
    "time"  
    "math"  
  
    "iceslide/src/board"  
    "iceslide/src/heuristic"  
    "iceslide/src/state"  
    "iceslide/src/successor"  
)  
  
type Result struct {  
    Goal *Node  
    Path []*Node  
    Cost int  
    NodesExpanded int  
    NodesGenerated int  
    Duration time.Duration  
}  
  
type Config struct {  
    Board *board.Board  
    Start state.State  
    Algorithm Algorithm  
    Heuristic heuristic.Func  
    Generator successor.Generator  
}  
  
func Search(cfg Config) (*Result, error) {  
    if cfg.Board == nil {  
        return nil, errors.New("search: nil board")  
    }  
    if cfg.Generator == nil {  
        return nil, errors.New("search: nil generator")  
    }  
    if cfg.Heuristic == nil {  
        return nil, errors.New("search: nil heuristic")  
    }  
}
```

```

    }
    if cfg.Algorithm == IDAStar {
        return searchIDAStar(cfg)
    }

    startTime := time.Now()

    root := &Node{
        State: cfg.Start,
        G: 0,
        H: cfg.Heuristic(cfg.Start, cfg.Board),
    }

    pq := newPriorityQueue(priorityFor(cfg.Algorithm))
    heap.Push(pq, root)

    bestG := map[uint64]int{cfg.Start.Key(): 0}
    expanded := 0
    generated := 1

    for pq.Len() > 0 {
        node := heap.Pop(pq).(*Node)

        if g, ok := bestG[node.State.Key()]; ok && g < node.G {
            continue
        }

        if isGoal(cfg.Board, node) {
            return &Result{
                Goal: node,
                Path: node.Path(),
                Cost: node.G,
                NodesExpanded: expanded,
                NodesGenerated: generated,
                Duration: time.Since(startTime),
            }, nil
        }
    }

```

```

        expanded++

        for _, succ := range cfg.Generator.Generate(cfg.Board,
node.State) {
            childG := node.G + succ.StepCost
            childKey := succ.State.Key()

            if g, ok := bestG[childKey]; ok && g <= childG {
                continue
            }
            bestG[childKey] = childG

            child := &Node{
                State: succ.State,
                Parent: node,
                Action: succ.Direction,
                G: childG,
                H: cfg.Heuristic(succ.State, cfg.Board),
            }
            heap.Push(pq, child)
            generated++
        }
    }

    return &Result{
        Goal: nil,
        Path: nil,
        Cost: 0,
        NodesExpanded: expanded,
        NodesGenerated: generated,
        Duration: time.Since(startTime),
    }, nil
}

func searchIDAStar(cfg Config) (*Result, error) {
    startTime := time.Now()

    root := &Node{

```

```

        State: cfg.Start,
        G: 0,
        H: cfg.Heuristic(cfg.Start, cfg.Board),
    }

    bound := root.H

    expanded := 0
    generated := 1
    pathVisited := make(map[uint64]bool)

    var dfs func(node *Node, currentBound int) (bool, int, *Node)
    dfs = func(node *Node, currentBound int) (bool, int, *Node) {
        f := node.G + node.H

        if f > currentBound {
            return false, f, nil
        }
        if isGoal(cfg.Board, node) {
            return true, f, node
        }

        expanded++
        minOutlier := math.MaxInt32
        pathVisited[node.State.Key()] = true

        for _, succ := range cfg.Generator.Generate(cfg.Board,
node.State) {
            if pathVisited[succ.State.Key()] {
                continue
            }

            child := &Node{
                State: succ.State,
                Parent: node,
                Action: succ.Direction,
                G: node.G + succ.StepCost,
                H: cfg.Heuristic(succ.State, cfg.Board),
            }

```

```

    }
    generated++

    found, newBound, goalNode := dfs(child, currentBound)
    if found {
        return true, newBound, goalNode
    }
    if newBound < minOutlier {
        minOutlier = newBound
    }
}

delete(pathVisited, node.State.Key())
return false, minOutlier, nil
}

for {
    found, newBound, goalNode := dfs(root, bound)
    if found {
        return &Result{
            Goal: goalNode,
            Path: goalNode.Path(),
            Cost: goalNode.G,
            NodesExpanded: expanded,
            NodesGenerated: generated,
            Duration: time.Since(startTime),
        }, nil
    }

    if newBound == math.MaxInt32 {
        return &Result{
            Goal: nil,
            Path: nil,
            Cost: 0,
            NodesExpanded: expanded,
            NodesGenerated: generated,
            Duration: time.Since(startTime),
        }, nil
    }
}

```

	<pre> } bound = newBound } } func isGoal(b *board.Board, n *Node) bool { if !b.InBounds(n.State.X, n.State.Y) { return false } cell := b.At(n.State.X, n.State.Y) return cell.Type == board.CellExit && n.State.NextDigit > b.MaxDigit() } </pre>
visualizer.go	<pre> package visualizer import ("bufio" "fmt" "io" "os" "strconv" "strings" "iceslide/src/board" "iceslide/src/heuristic" "iceslide/src/search" "iceslide/src/state" "iceslide/src/successor") type Frame struct { Index int Pos state.State Action string StepCost int TotalCost int </pre>

```

        Collected []bool
    }

func BuildFrames(b *board.Board, path []*search.Node) []Frame {
    if len(path) == 0 {
        return nil
    }
    frames := make([]Frame, len(path))
    for i, node := range path {
        action := "Initial"
        stepCost := 0
        if i > 0 {
            action = directionName(node.Action)
            stepCost = node.G - path[i-1].G
        }
        frames[i] = Frame{
            Index:      i,
            Pos:         node.State,
            Action:      action,
            StepCost:    stepCost,
            TotalCost:   node.G,
            Collected:  collectedSet(node.State.NextDigit,
b.MaxDigit()),
        }
    }
    return frames
}

func PrintSummary(w io.Writer, alg search.Algorithm, hName string,
b *board.Board, start state.State, res *search.Result) {
    fmt.Fprintln(w)
    fmt.Fprintln(w, "== Initial Board ==")
    fmt.Fprint(w, RenderInitial(b, start))

    fmt.Fprintln(w)
    fmt.Fprintln(w, "== Solution Summary ==")
    fmt.Fprintf(w, "Algorithm      : %s\n", alg)
    fmt.Fprintf(w, "Heuristic      : %s\n", hName)
}

```

```

    if res.Goal == nil {
        fmt.Fprintln(w, "Result          : NO PATH FOUND")
    } else {
        fmt.Fprintf(w, "Path          : %s\n",
PathString(res.Path))
        fmt.Fprintf(w, "Cost          : %d\n", res.Cost)
        fmt.Fprintf(w, "Steps         : %d\n", len(res.Path)-1)
    }
    fmt.Fprintf(w, "Iterations     : %d\n", res.NodesExpanded)
    fmt.Fprintf(w, "Nodes generated : %d\n", res.NodesGenerated)
    fmt.Fprintf(w, "Waktu eksekusi : %d ms\n",
res.Duration.Milliseconds())
    fmt.Fprintln(w)
}

func PathString(path []*search.Node) string {
    if len(path) <= 1 {
        return "(no moves)"
    }
    parts := make([]string, 0, len(path)-1)
    for i := 1; i < len(path); i++ {
        parts = append(parts, path[i].Action.String())
    }
    return strings.Join(parts, "")
}

func RenderInitial(b *board.Board, s state.State) string {
    return renderBoardAt(b, s, collectedSet(0, b.MaxDigit()))
}

func RenderFrame(b *board.Board, f Frame) string {
    return renderBoardAt(b, f.Pos, f.Collecte d)
}

func Playback(scanner *bufio.Scanner, w io.Writer, b *board.Board,
path []*search.Node) error {
    frames := BuildFrames(b, path)

```

```

if len(frames) == 0 {
    fmt.Fprintln(w, "(tidak ada frame untuk diputar)")
    return nil
}

idx := 0
for {
    printFrame(w, b, frames[idx], len(frames)-1)
    fmt.Fprint(w, "[n/Enter] next  [p] prev  [j] jump  [q]
keluar > ")

    if !scanner.Scan() {
        return scanner.Err()
    }
    cmd := strings.ToLower(strings.TrimSpace(scanner.Text()))

    switch cmd {
    case "n", "":
        if idx < len(frames)-1 {
            idx++
        } else {
            fmt.Fprintln(w, "(sudah di langkah terakhir)")
        }
    case "p":
        if idx > 0 {
            idx--
        } else {
            fmt.Fprintln(w, "(sudah di langkah pertama)")
        }
    case "j":
        fmt.Fprint(w, "Pada step berapa anda ingin melakukan
        playback : ")
        if !scanner.Scan() {
            return scanner.Err()
        }

        target, err :=
        strconv.Atoi(strings.TrimSpace(scanner.Text()))
        if err != nil || target < 0 || target > len(frames)-1 {

```

```

        fmt.Fprintf(w, "(input tidak valid; range
0..%d)\n", len(frames)-1)
        continue
    }
    idx = target
case "q":
    return nil
default:
    fmt.Fprintln(w, "(perintah tidak dikenal)")
}
}

func PromptYesNo(scanner *bufio.Scanner, w io.Writer, prompt
string) (bool, error) {
    for {
        fmt.Fprintf(w, "%s (Ya/Tidak): ", prompt)
        if !scanner.Scan() {
            return false, scanner.Err()
        }
        switch strings.ToLower(strings.TrimSpace(scanner.Text())) {
        case "ya", "y", "yes":
            return true, nil
        case "tidak", "t", "n", "no", "":
            return false, nil
        default:
            fmt.Fprintln(w, "(jawab dengan Ya atau Tidak)")
        }
    }
}

func PromptLine(scanner *bufio.Scanner, w io.Writer, prompt string)
(string, error) {
    fmt.Fprint(w, prompt)
    if !scanner.Scan() {
        return "", scanner.Err()
    }
    return strings.TrimSpace(scanner.Text()), nil
}

```

```

}

func SelectAlgorithm(scanner *bufio.Scanner, w io.Writer)
(search.Algorithm, error) {
    for {
        fmt.Fprintln(w)
        fmt.Fprintln(w, "== Pilih Algoritma ==")
        fmt.Fprintln(w, " 1. Uniform Cost Search (UCS)")
        fmt.Fprintln(w, " 2. Greedy Best-First Search (GBFS)")
        fmt.Fprintln(w, " 3. A*")
        fmt.Fprintln(w, " 4. Iterative Deepening A* (IDA*)")
        fmt.Fprint(w, "Pilihan [1-4]: ")
        if !scanner.Scan() {
            return 0, scanner.Err()
        }
        switch strings.TrimSpace(scanner.Text()) {
        case "1":
            return search.UCS, nil
        case "2":
            return search.GBFS, nil
        case "3":
            return search.AStar, nil
        case "4":
            return search.IDAStar, nil
        default:
            fmt.Fprintln(w, "(input tidak valid; pilih 1, 2, 3,
atau 4)")
        }
    }
}

func SelectHeuristic(scanner *bufio.Scanner, w io.Writer, alg
search.Algorithm) (heuristic.Func, string, error) {
    if alg == search.UCS {
        return heuristic.Zero, "Zero (UCS)", nil
    }
    for {
        fmt.Fprintln(w)

```

```

        fmt.Fprintln(w, "== Pilih Heuristik ==")
        fmt.Fprintln(w, "  1. Manhattan")
        fmt.Fprintln(w, "  2. Remaining Digits")
        fmt.Fprintln(w, "  3. Euclidean")
        fmt.Fprintln(w, "  4. Max Combined (max of Manhattan &
Remaining Digits)")
        fmt.Fprint(w, "Pilihan [1-4]: ")
        if !scanner.Scan() {
            return nil, "", scanner.Err()
        }
        switch strings.TrimSpace(scanner.Text()) {
        case "1":
            return heuristic.Manhattan, "Manhattan", nil
        case "2":
            return heuristic.RemainingDigits, "RemainingDigits",
nil
        case "3":
            return heuristic.Euclidean, "Euclidean", nil
        case "4":
            return heuristic.MaxCombined, "Max Combined", nil
        default:
            fmt.Fprintln(w, "(input tidak valid; pilih 1 atau 2)")
        }
    }
}

func RunInteractive(
    scanner *bufio.Scanner,
    w io.Writer,
    alg search.Algorithm,
    hName string,
    b *board.Board,
    start state.State,
    res *search.Result,
    defaultSavePath string,
) error {
    if res == nil || res.Goal == nil {
        return nil
    }

```

```

    }

    play, err := PromptYesNo(scanner, w, "Apakah Anda ingin
melakukan playback?")
    if err != nil {
        return err
    }
    if play {
        if err := Playback(scanner, w, b, res.Path); err != nil {
            return err
        }
    }
}

    save, err := PromptYesNo(scanner, w, "Apakah Anda ingin
menyimpan solusi?")
    if err != nil {
        return err
    }
    if !save {
        return nil
    }

    outPath, err := PromptLine(scanner, w,
        fmt.Sprintf("Nama file output [%s]: ", defaultSavePath))
    if err != nil {
        return err
    }
    if outPath == "" {
        outPath = defaultSavePath
    }

    if err := SaveSolution(outPath, alg, hName, b, start, res); err
!= nil {
        return fmt.Errorf("save %s: %w", outPath, err)
    }
    fmt.Fprintf(w, "Solusi disimpan pada %s\n", outPath)
    return nil
}

```

```

func SaveSolution(path string, alg search.Algorithm, hName string,
b *board.Board, start state.State, res *search.Result) error {
    f, err := os.Create(path)
    if err != nil {
        return err
    }
    defer f.Close()

    bw := bufio.NewWriter(f)
    defer bw.Flush()

    PrintSummary(bw, alg, hName, b, start, res)

    if res.Goal == nil {
        return nil
    }

    fmt.Fprintln(bw)
    fmt.Fprintln(bw, "== Step-by-step ==")
    for _, fr := range BuildFrames(b, res.Path) {
        fmt.Fprintln(bw)
        printFrame(bw, b, fr, len(res.Path)-1)
    }
    return nil
}

func printFrame(w io.Writer, b *board.Board, f Frame, last int) {
    fmt.Fprintf(w, "\n--- Step %d / %d (%s) ---\n", f.Index, last,
f.Action)
    fmt.Fprint(w, RenderFrame(b, f))
    if f.Index > 0 {
        fmt.Fprintf(w, "Move: %s    step cost: %d    total cost:
%d\n",
            f.Action, f.StepCost, f.TotalCost)
    }
}

```

```

func renderBoardAt(b *board.Board, pos state.State, collected
[]bool) string {
    var sb strings.Builder
    for y := 0; y < b.Height(); y++ {
        for x := 0; x < b.Width(); x++ {
            if x == pos.X && y == pos.Y {
                sb.WriteRune('Z')
                continue
            }
            c := b.At(x, y)
            if c.Type == board.CellNumber && c.Digit <
len(collected) && collected[c.Digit] {
                sb.WriteRune('*')
                continue
            }
            sb.WriteRune(glyphFor(c))
        }
        sb.WriteByte('\n')
    }
    return sb.String()
}

func collectedSet(nextDigit, maxDigit int) []bool {
    size := maxDigit + 1
    if size < 0 {
        size = 0
    }
    out := make([]bool, size)
    for d := 0; d < nextDigit && d < size; d++ {
        out[d] = true
    }
    return out
}

func glyphFor(c board.Cell) rune {
    switch c.Type {
    case board.CellRock:
        return 'X'
    }
}

```

	<pre>case board.CellLava: return 'L' case board.CellExit: return 'O' case board.CellNumber: return rune('0' + c.Digit) default: return '*' } }</pre> <pre>func directionName(d successor.Direction) string { switch d { case successor.Up: return "Up" case successor.Down: return "Down" case successor.Left: return "Left" case successor.Right: return "Right" default: return "?" } }</pre>
app.go	<pre>package gui import ("encoding/json" "fmt" "io" "net/http" "os/exec" "runtime" "strings" "time"</pre>

```

    "iceslide/src/board"
    "iceslide/src/heuristic"
    "iceslide/src/search"
    "iceslide/src/state"
    "iceslide/src/successor"
    "iceslide/src/visualizer"
)

func Run() {
    mux := http.NewServeMux()
    mux.HandleFunc("/", serveIndex)
    mux.HandleFunc("/api/solve", handleSolve)

    addr := "127.0.0.1:8765"
    fmt.Println("GUI berjalan di http://" + addr)
    fmt.Println("Tekan Ctrl+C untuk keluar.")

    go func() {
        time.Sleep(300 * time.Millisecond)
        openBrowser("http://" + addr)
    }()

    if err := http.ListenAndServe(addr, mux); err != nil {
        fmt.Println("server error:", err)
    }
}

func openBrowser(url string) {
    switch runtime.GOOS {
    case "windows":
        exec.Command("cmd", "/c", "start", url).Start()
    case "darwin":
        exec.Command("open", url).Start()
    default:
        exec.Command("xdg-open", url).Start()
    }
}

```

```

type cellJSON struct {
    Type string `json:"t"`
    Digit int    `json:"d"`
    Cost  int    `json:"c"`
}

type frameJSON struct {
    Index      int    `json:"index"`
    Action     string `json:"action"`
    StepCost   int    `json:"stepCost"`
    TotalCost  int    `json:"totalCost"`
    PX         int    `json:"px"`
    PY         int    `json:"py"`
    Collected []bool `json:"collected"`
}

type solveResponse struct {
    Found          bool    `json:"found"`
    Path           string  `json:"path"`
    Cost           int     `json:"cost"`
    Steps          int     `json:"steps"`
    Iterations     int     `json:"iterations"`
    NodesGenerated int     `json:"nodesGenerated"`
    DurationMs     float64 `json:"durationMs"`
    Algorithm      string  `json:"algorithm"`
    Heuristic      string  `json:"heuristic"`
    Width          int     `json:"width"`
    Height         int     `json:"height"`
    Cells          [][]cellJSON `json:"cells"`
    Frames         []frameJSON `json:"frames"`
    Error          string    `json:"error,omitempty"`
}

func handleSolve(w http.ResponseWriter, r *http.Request) {
    w.Header().Set("Content-Type", "application/json")
    w.Header().Set("Access-Control-Allow-Origin", "*")

    if r.Method == http.MethodOptions {

```

```

        w.WriteHeader(http.StatusOK)
        return
    }
    if r.Method != http.MethodPost {
        http.Error(w, "method not allowed",
http.StatusMethodNotAllowed)
        return
    }

    if err := r.ParseMultipartForm(10 << 20); err != nil {
        jsonError(w, "parse form: "+err.Error())
        return
    }

    file, _, err := r.FormFile("file")
    if err != nil {
        jsonError(w, "baca file: "+err.Error())
        return
    }
    defer file.Close()

    pr, err := board.Parse(file)
    if err != nil {
        jsonError(w, "parse board: "+err.Error())
        return
    }

    algStr := r.FormValue("alg")
    heurStr := r.FormValue("heur")

    alg := parseAlg(algStr)
    h, hLabel := parseHeuristic(alg, heurStr)

    start := state.State{X: pr.StartX, Y: pr.StartY, NextDigit: 0}

    res, err := search.Search(search.Config{
        Board:    pr.Board,
        Start:    start,

```

```

        Algorithm: alg,
        Heuristic: h,
        Generator: successor.SlideGenerator{},
    })
    if err != nil {
        jsonError(w, "search: "+err.Error())
        return
    }

    b := pr.Board
    cells := make([][]cellJSON, b.Height())
    for y := 0; y < b.Height(); y++ {
        cells[y] = make([]cellJSON, b.Width())
        for x := 0; x < b.Width(); x++ {
            c := b.At(x, y)
            cells[y][x] = cellJSON{
                Type: cellTypeName(c.Type),
                Digit: c.Digit,
                Cost:  c.Cost,
            }
        }
    }

    resp := solveResponse{
        Algorithm:    alg.String(),
        Heuristic:     hLabel,
        Width:         b.Width(),
        Height:        b.Height(),
        Cells:         cells,
        Iterations:    res.NodesExpanded,
        NodesGenerated: res.NodesGenerated,
        DurationMs:    float64(res.Duration.Microseconds()) /
1000.0,
    }

    if res.Goal != nil {
        resp.Found = true
        resp.Path = visualizer.PathString(res.Path)
    }

```

```

        resp.Cost = res.Cost
        resp.Steps = len(res.Path) - 1

        frames := visualizer.BuildFrames(b, res.Path)
        resp.Frames = make([]frameJSON, len(frames))
        for i, f := range frames {
            resp.Frames[i] = frameJSON{
                Index:    f.Index,
                Action:    f.Action,
                StepCost:  f.StepCost,
                TotalCost: f.TotalCost,
                PX:        f.Pos.X,
                PY:        f.Pos.Y,
                Collected: f.Collected,
            }
        }
    }

    json.NewEncoder(w).Encode(resp)
}

func jsonError(w http.ResponseWriter, msg string) {
    resp := solveResponse{Error: msg}
    json.NewEncoder(w).Encode(resp)
}

func cellTypeName(ct board.CellType) string {
    switch ct {
    case board.CellRock:
        return "rock"
    case board.CellLava:
        return "lava"
    case board.CellExit:
        return "exit"
    case board.CellNumber:
        return "number"
    default:
        return "ice"
    }
}

```

```

    }
}

func parseAlg(s string) search.Algorithm {
    switch strings.ToLower(s) {
    case "ucs":
        return search.UCS
    case "gbfs":
        return search.GBFS
    case "idastar", "ida*":
        return search.IDAStar
    default:
        return search.AStar
    }
}

func parseHeuristic(alg search.Algorithm, s string)
(heuristic.Func, string) {
    if alg == search.UCS {
        return heuristic.Zero, "Zero (UCS)"
    }
    switch strings.ToLower(s) {
    case "remaining":
        return heuristic.RemainingDigits, "Remaining Digits"
    case "euclidean":
        return heuristic.Euclidean, "Euclidean"
    case "maxcombined":
        return heuristic.MaxCombined, "Max Combined"
    default:
        return heuristic.Manhattan, "Manhattan"
    }
}

func serveIndex(w http.ResponseWriter, r *http.Request) {
    w.Header().Set("Content-Type", "text/html; charset=utf-8")
    io.WriteString(w, indexHTML)
}

```

index.go

```
package gui
var indexHTML = `<!DOCTYPE html>
<html lang="id">
<head>
<meta charset="UTF-8">
<title>Ice Sliding Puzzle Solver</title>
<style>
*{box-sizing:border-box;margin:0;padding:0}
body{background:#0f172a;color:#e2e8f0;font-family:'Segoe
UI',system-ui,sans-serif;height:100vh;display:flex;flex-direction:c
olumn;overflow:hidden}
header{background:#1e293b;padding:12px 20px;border-bottom:1px solid
#334155;display:flex;align-items:center;gap:12px;flex-shrink:0}
header h1{font-size:18px;font-weight:700;color:#38bdf8}
header span{color:#64748b;font-size:13px}
.main{display:flex;flex:1;overflow:hidden}
.left{width:300px;background:#1e293b;border-right:1px solid
#334155;overflow-y:auto;flex-shrink:0;padding:16px;display:flex;fle
x-direction:column;gap:12px}
.right{flex:1;display:flex;flex-direction:column;overflow:hidden}
.board-area{flex:1;overflow:auto;padding:16px;display:flex;align-it
ems:flex-start;justify-content:flex-start}
.controls{background:#1e293b;border-top:1px solid
#334155;padding:12px 16px;flex-shrink:0}

label{font-size:12px;font-weight:600;color:#94a3b8;text-transform:u
ppercase;letter-spacing:.05em;display:block;margin-bottom:4px}
select,input[type=file]{width:100%;background:#0f172a;border:1px
solid #334155;color:#e2e8f0;padding:7px
10px;border-radius:6px;font-size:13px}
select:focus{outline:none;border-color:#38bdf8}
input[type=range]{width:100%;accent-color:#38bdf8}

.btn{width:100%;padding:9px;border:none;border-radius:6px;font-size
:14px;font-weight:600;cursor:pointer;transition:opacity .15s}
.btn-primary{background:#0284c7;color:#fff}
.btn-primary:hover{opacity:.85}
.btn-primary:disabled{background:#334155;color:#64748b;cursor:defau
```

```

lt;opacity:1}
.btn-secondary{background:#1e3a5f;color:#38bdf8;border:1px solid
#38bdf8}
.btn-secondary:hover{opacity:.85}
.btn-secondary:disabled{background:#1e293b;color:#64748b;border-col
or:#334155;cursor:default}

.section-title{font-size:12px;font-weight:700;color:#38bdf8;text-tr
ansform:uppercase;letter-spacing:.08em;padding-bottom:6px;border-bo
ttom:1px solid #334155}

.stats-grid{display:grid;grid-template-columns:auto 1fr;gap:4px
10px;font-size:12px}
.stats-grid .k{color:#64748b}
.stats-grid .v{color:#e2e8f0;font-weight:500;word-break:break-all}

.legend{display:grid;grid-template-columns:1fr
1fr;gap:4px;font-size:11px}
.leg-item{display:flex;align-items:center;gap:5px}
.swatch{width:14px;height:14px;border-radius:2px;flex-shrink:0}

.status{padding:6px
10px;border-radius:6px;font-size:12px;font-weight:600;text-align:ce
nter}
.status-idle{background:#1e293b;color:#64748b;border:1px solid
#334155}
.status-running{background:#1e3a5f;color:#38bdf8;border:1px solid
#0284c7}
.status-ok{background:#14532d;color:#4ade80;border:1px solid
#16a34a}
.status-fail{background:#450a0a;color:#f87171;border:1px solid
#dc2626}

#board{display:grid;gap:1px;background:#0f172a}
.cell{width:46px;height:46px;display:flex;align-items:center;justif
y-content:center;font-weight:700;font-size:16px;border-radius:3px;t
ransition:background .2s}
.cell-ice{background:#bfbdbf}

```

```

.cell-rock{background:#374151;color:#9ca3af}
.cell-lava{background:#dc2626;color:#fff}
.cell-exit{background:#16a34a;color:#fff}
.cell-number{background:#fbbf24;color:#1c1917}
.cell-collected{background:#6b7280;color:#e5e7eb}
.cell-player{background:#2563eb;color:#fff;box-shadow:0 0 0 2px
#93c5fd}
.cell-path{background:#93c5fd}

.pb-row{display:flex;align-items:center;gap:10px;margin-bottom:8px}
.pb-btn{background:#1e293b;border:1px solid #334155;color:#e2e8f0;width:34px;height:34px;border-radius:6px;cursor:pointer;display:flex;align-items:center;justify-content:center;font-size:14px;transition:background .15s}
.pb-btn:hover{background:#334155}
.pb-btn:disabled{opacity:.4;cursor:default}
#step-info{font-size:12px;color:#94a3b8;flex:1;text-align:center}
input[type=range].step-range{flex:1;height:6px}
.speed-wrap{display:flex;align-items:center;gap:8px;font-size:12px;color:#94a3b8}
#speed-val{color:#38bdf8;min-width:36px;font-weight:600}

.file-drop{border:2px dashed #334155;border-radius:8px;padding:14px;text-align:center;font-size:12px;color:#64748b;cursor:pointer;transition:border-color .2s}
.file-drop:hover,.file-drop.drag-over{border-color:#38bdf8;color:#38bdf8}
</style>
</head>
<body>
<header>
  <h1>Ice Sliding Puzzle Solver</h1>
  <span>IF2211 Strategi Algoritma</span>
</header>
<div class="main">
  <div class="left">
    <div class="section-title">Input</div>
    <div>

```

```

<label>File Puzzle (.txt)</label>
    <div class="file-drop" id="drop-zone"
onclick="document.getElementById('file-input').click()">
        Klik atau drag & drop file .txt
        <br><span id="file-name"
style="color:#38bdf8;font-weight:600"></span>
    </div>
    <input type="file" id="file-input" accept=".txt"
style="display:none" onchange="onFileChange(this)">
</div>
<div>
    <label>Algoritma</label>
    <select id="alg-select" onchange="onAlgChange()">
        <option value="astar">A*</option>
        <option value="ucs">UCS</option>
        <option value="gbfs">GBFS</option>
        <option value="idastar">IDA*</option>
    </select>
</div>
<div>
    <label>Heuristik</label>
    <select id="heur-select">
        <option value="manhattan">Manhattan</option>
        <option value="remaining">Remaining Digits</option>
        <option value="euclidean">Euclidean</option>
        <option value="maxcombined">Max Combined</option>
    </select>
</div>
    <button class="btn btn-primary" id="run-btn"
onclick="runSolve()" disabled>Cari Solusi</button>
    <div class="status status-idle" id="status-box">Pilih file
input</div>

<div class="section-title">Hasil</div>
<div class="stats-grid">
    <span class="k">Algoritma</span><span class="v"
id="s-alg">-</span>
    <span class="k">Heuristik</span><span class="v"

```

```

id="s-heur">-</span>
            <span    class="k">Cost</span><span    class="v"
id="s-cost">-</span>
            <span    class="k">Steps</span><span    class="v"
id="s-steps">-</span>
            <span    class="k">Iterasi</span><span    class="v"
id="s-iter">-</span>
            <span    class="k">Nodes</span><span    class="v"
id="s-nodes">-</span>
            <span    class="k">Durasi    (ms)</span><span    class="v"
id="s-dur">-</span>
            <span    class="k">Path</span><span    class="v" id="s-path"
style="font-family:monospace">-</span>
        </div>

        <button    class="btn    btn-secondary"    id="save-btn"
onclick="saveSolution()"    disabled>Simpan Solusi (.txt)</button>

        <div    class="section-title">Legenda</div>
        <div    class="legend">
            <div    class="leg-item"><div    class="swatch"
style="background:#2563eb"></div><span>Z = Pemain</span></div>
            <div    class="leg-item"><div    class="swatch"
style="background:#16a34a"></div><span>O = Tujuan</span></div>
            <div    class="leg-item"><div    class="swatch"
style="background:#374151"></div><span>X = Batu</span></div>
            <div    class="leg-item"><div    class="swatch"
style="background:#dc2626"></div><span>L = Lava</span></div>
            <div    class="leg-item"><div    class="swatch"
style="background:#fbbf24"></div><span>0-9 = Digit</span></div>
            <div    class="leg-item"><div    class="swatch"
style="background:#6b7280"></div><span>* = Terlewati</span></div>
            <div    class="leg-item"><div    class="swatch"
style="background:#bfdbfe"></div><span>Es</span></div>
            <div    class="leg-item"><div    class="swatch"
style="background:#93c5fd"></div><span>Jejak</span></div>
        </div>
    </div>

```

```

<div class="right">
  <div class="board-area">
    <div id="board"></div>
  </div>
  <div class="controls">
    <div class="pb-row">
      <button class="pb-btn" id="pb-first" onclick="pbFirst()"
title="Awal" disabled>&#x23EE;</button>
      <button class="pb-btn" id="pb-prev"  onclick="pbPrev()"
title="Mundur" disabled>&#x25C0;</button>
      <button class="pb-btn" id="pb-play"  onclick="pbPlay()"
title="Play/Pause" disabled>&#x25B6;</button>
      <button class="pb-btn" id="pb-next"  onclick="pbNext()"
title="Maju" disabled>&#x25B6;&#x7C;</button>
      <button class="pb-btn" id="pb-last"  onclick="pbLast()"
title="Akhir" disabled>&#x23ED;</button>
      <div class="speed-wrap">
        <span>Kecepatan:</span>
        <input type="range" id="speed-range" min="0.25" max="4"
step="0.25" value="1" oninput="onSpeedChange(this.value)"
style="width:100px">
        <span id="speed-val">1.00x</span>
      </div>
    </div>
    <div class="pb-row">
      <input type="range" class="step-range" id="step-range"
min="0" max="0" value="0" oninput="pbJump(parseInt(this.value))">
    </div>
    <div id="step-info">Step: - / -</div>
  </div>
</div>

<script>
let state = {
  file: null,
  data: null,

```

```

    curFrame: 0,
    playing: false,
    timer: null,
    speed: 1.0,
    visitedPos: []
  };

function onFileChange(input) {
  if (input.files.length === 0) return;
  state.file = input.files[0];
  document.getElementById('file-name').textContent =
state.file.name;
  document.getElementById('run-btn').disabled = false;
  setStatus('idle', 'File dipilih: ' + state.file.name);
}

const dz = document.getElementById('drop-zone');
dz.addEventListener('dragover', e => { e.preventDefault();
dz.classList.add('drag-over'); });
dz.addEventListener('dragleave', () => {
dz.classList.remove('drag-over');
dz.addEventListener('drop', e => {
  e.preventDefault(); dz.classList.remove('drag-over');
  const f = e.dataTransfer.files[0];
  if (!f) return;
  state.file = f;
  document.getElementById('file-name').textContent = f.name;
  document.getElementById('run-btn').disabled = false;
  setStatus('idle', 'File dipilih: ' + f.name);
});

function onAlgChange() {
  const alg = document.getElementById('alg-select').value;
  document.getElementById('heur-select').disabled = alg === 'ucs';
}

function setStatus(type, msg) {
  const el = document.getElementById('status-box');

```

```

    el.className = 'status status-' + type;
    el.textContent = msg;
  }

  async function runSolve() {
    if (!state.file) return;
    stopPlay();
    setStatus('running', 'Memproses...');
    document.getElementById('run-btn').disabled = true;
    document.getElementById('save-btn').disabled = true;

    const fd = new FormData();
    fd.append('file', state.file);
    fd.append('alg', document.getElementById('alg-select').value);
    fd.append('heur', document.getElementById('heur-select').value);

    try {
      const resp = await fetch('/api/solve', { method: 'POST', body: fd });
      const data = await resp.json();
      if (data.error) { setStatus('fail', 'Error: ' + data.error); return; }
      state.data = data;
      state.curFrame = 0;
      state.visitedPos = [];
      updateStats(data);
      if (!data.found) {
        setStatus('fail', 'Tidak ada solusi ditemukan.');
```

```

        renderFrame(0);
      } else {
        setStatus('ok', 'Solusi ditemukan! ' + data.steps + ' langkah, cost: ' + data.cost);
        initPlayback(data);
        renderFrame(0);
        document.getElementById('save-btn').disabled = false;
      }
    } catch(e) {
      setStatus('fail', 'Network error: ' + e.message);
    }
  }

```

```

    } finally {
        document.getElementById('run-btn').disabled = false;
    }
}

function updateStats(d) {
    document.getElementById('s-alg').textContent = d.algorithm;
    document.getElementById('s-heur').textContent = d.heuristic;
    document.getElementById('s-cost').textContent = d.found ? d.cost
: '-';
    document.getElementById('s-steps').textContent = d.found ?
d.steps : '-';
    document.getElementById('s-iter').textContent = d.iterations;
    document.getElementById('s-nodes').textContent =
d.nodesGenerated;
    document.getElementById('s-dur').textContent =
d.durationMs.toFixed(3);
    document.getElementById('s-path').textContent = d.found ? d.path
: '-';
}

function initPlayback(data) {
    const n = data.frames.length - 1;
    const sr = document.getElementById('step-range');
    sr.max = n; sr.value = 0;
    setPbBtns(true);
}

function setPbBtns(enabled) {
    ['pb-first', 'pb-prev', 'pb-play', 'pb-next', 'pb-last'].forEach(id
=> {
        document.getElementById(id).disabled = !enabled;
    });
}

function renderFrame(idx) {
    const d = state.data;
    if (!d) return;

```

```

const frames = d.frames || [];
if (frames.length === 0) return;
if (idx < 0) idx = 0;
if (idx >= frames.length) idx = frames.length - 1;
state.curFrame = idx;

const fr = frames[idx];
document.getElementById('step-range').value = idx;

const action = fr.action === 'Initial' ? 'Initial' : fr.action;
document.getElementById('step-info').textContent =
  'Step ' + idx + ' / ' + (frames.length - 1) +
  ' | Aksi: ' + action +
  (idx > 0 ? ' | Step Cost: ' + fr.stepCost + ' | Total Cost: ' + fr.totalCost : '');

state.visitedPos = [];
for (let i = 0; i <= idx; i++) {
  state.visitedPos.push([d.frames[i].px, d.frames[i].py]);
}

const grid = document.getElementById('board');
grid.style.gridTemplateColumns = 'repeat(' + d.width + ', 46px)';
grid.innerHTML = '';

for (let y = 0; y < d.height; y++) {
  for (let x = 0; x < d.width; x++) {
    const cell = d.cells[y][x];
    const div = document.createElement('div');
    div.className = 'cell';

    let cls, txt = '';
    if (x === fr.px && y === fr.py) {
      cls = 'cell-player'; txt = 'Z';
    } else {
      switch (cell.t) {
        case 'rock': cls = 'cell-rock'; txt = 'X'; break;
        case 'lava': cls = 'cell-lava'; txt = 'L'; break;
      }
    }
  }
}

```

```

        case 'exit': cls = 'cell-exit'; txt = '0'; break;
        case 'number':
            const col = fr.collected && cell.d <
fr.collected.length && fr.collected[cell.d];
            cls = col ? 'cell-collected' : 'cell-number';
            txt = col ? '*' : String(cell.d);
            break;
        default:
            const isVisited = state.visitedPos.some(p => p[0] === x
&& p[1] === y);
            cls = isVisited ? 'cell-path' : 'cell-ice';
        }
    }
    div.classList.add(cls);
    div.textContent = txt;
    div.title = cell.t + (cell.t === 'number' ? ' ' + cell.d :
'') + ' (cost:' + cell.c + ')';
    grid.appendChild(div);
}
}
}

function pbFirst() { stopPlay(); renderFrame(0); }
function pbPrev() { stopPlay(); renderFrame(state.curFrame - 1); }
function pbNext() { stopPlay(); renderFrame(state.curFrame + 1); }
function pbLast() { stopPlay();
renderFrame(state.data.frames.length - 1); }
function pbJump(v) { stopPlay(); renderFrame(v); }

function pbPlay() {
    if (state.playing) { stopPlay(); return; }
    if (!state.data || !state.data.found) return;
    if (state.curFrame >= state.data.frames.length - 1)
renderFrame(0);
    state.playing = true;
    document.getElementById('pb-play').textContent = '⏮';
    const ms = 1000 / state.speed;
    state.timer = setInterval(() => {

```

```

        if (state.curFrame < state.data.frames.length - 1) {
            renderFrame(state.curFrame + 1);
        } else {
            stopPlay();
        }
    }, ms);
}

function stopPlay() {
    if (!state.playing) return;
    clearInterval(state.timer);
    state.playing = false;
    document.getElementById('pb-play').innerHTML = '&#x25B6;';
}

function onSpeedChange(v) {
    state.speed = parseFloat(v);
    document.getElementById('speed-val').textContent =
parseFloat(v).toFixed(2) + 'x';
    if (state.playing) { stopPlay(); pbPlay(); }
}

function saveSolution() {
    const d = state.data;
    if (!d || !d.found) return;
    let txt = '';
    txt += 'Algoritma      : ' + d.algorithm + '\n';
    txt += 'Heuristik       : ' + d.heuristic + '\n';
    txt += 'Path             : ' + d.path + '\n';
    txt += 'Cost             : ' + d.cost + '\n';
    txt += 'Steps            : ' + d.steps + '\n';
    txt += 'Iterations        : ' + d.iterations + '\n';
    txt += 'Nodes generated : ' + d.nodesGenerated + '\n';
    txt += 'Duration          : ' + d.durationMs.toFixed(3) + ' ms\n';
    txt += '\n== Step-by-step ==\n';
    d.frames.forEach(fr => {
        txt += '\n--- Step ' + fr.index + ' / ' + (d.frames.length-1) +
' (' + fr.action + ') ---\n';
    });
}

```

```

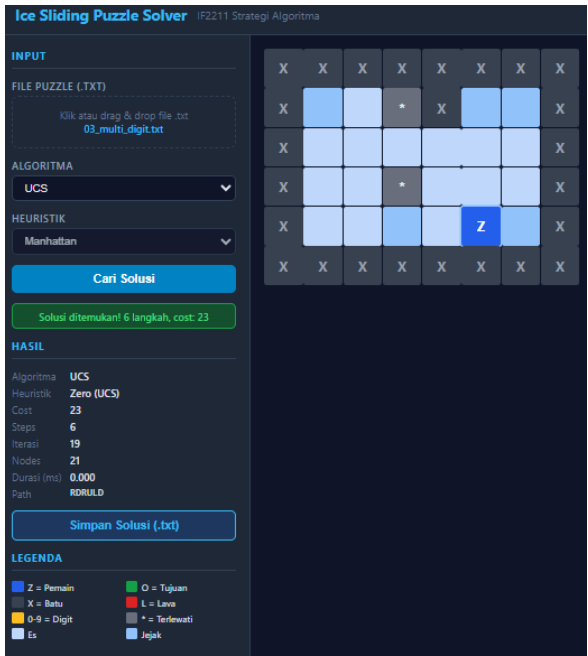
for (let y = 0; y < d.height; y++) {
  for (let x = 0; x < d.width; x++) {
    if (x === fr.px && y === fr.py) { txt += 'Z'; continue; }
    const c = d.cells[y][x];
    if (c.t === 'rock') { txt += 'X'; continue; }
    if (c.t === 'lava') { txt += 'L'; continue; }
    if (c.t === 'exit') { txt += 'O'; continue; }
    if (c.t === 'number') {
      const col = fr.collected && c.d < fr.collected.length &&
fr.collected[c.d];
      txt += col ? '*' : String(c.d); continue;
    }
    txt += '*';
  }
  txt += '\n';
}
if (fr.index > 0) txt += 'Move: ' + fr.action + '  step cost: '
+ fr.stepCost + '  total cost: ' + fr.totalCost + '\n';
});

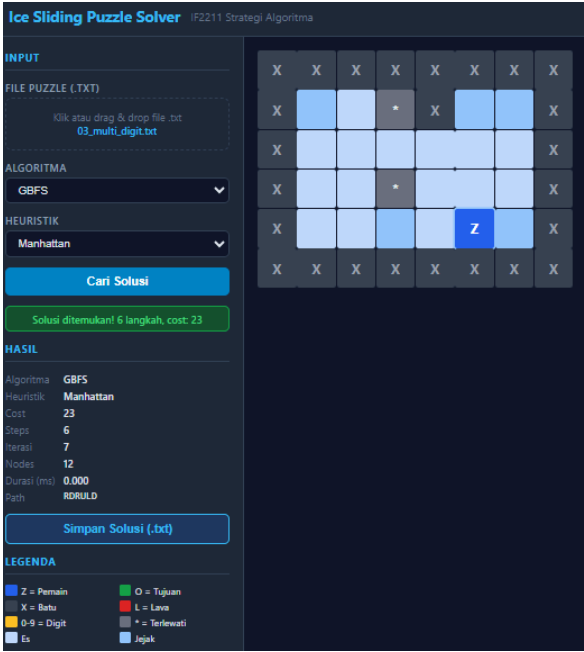
const blob = new Blob([txt], {type: 'text/plain'});
const a = document.createElement('a');
a.href = URL.createObjectURL(blob);
a.download = 'solusi.txt';
a.click();
}
</script>
</body>
</html>`

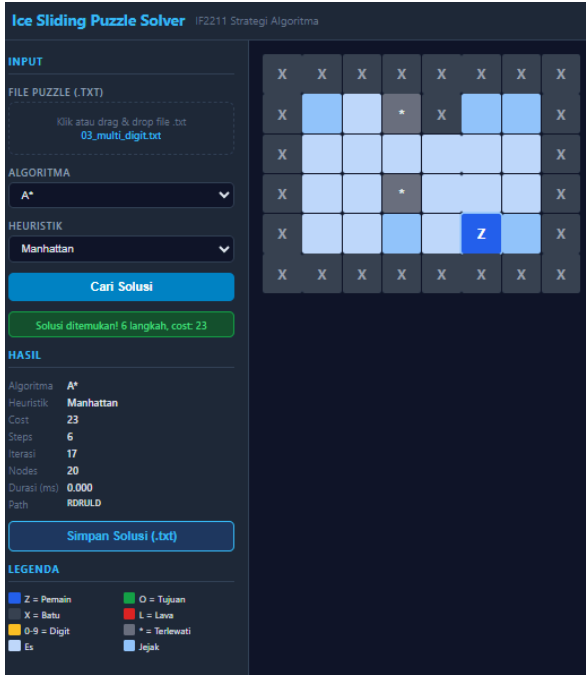
```

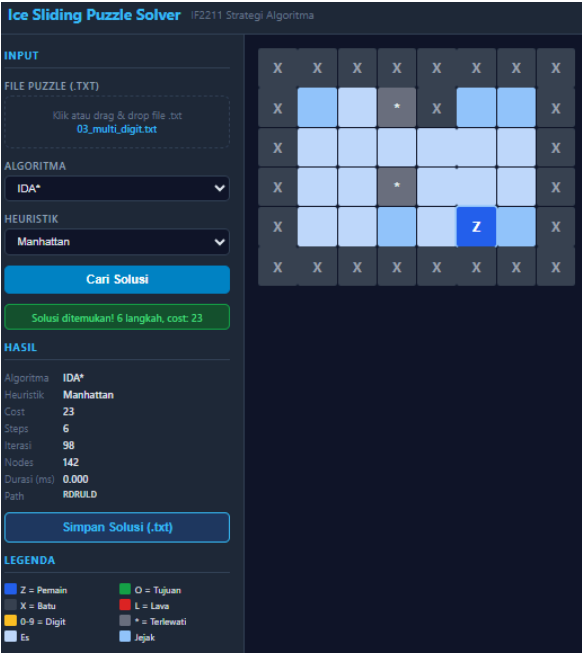
E. Hasil Program

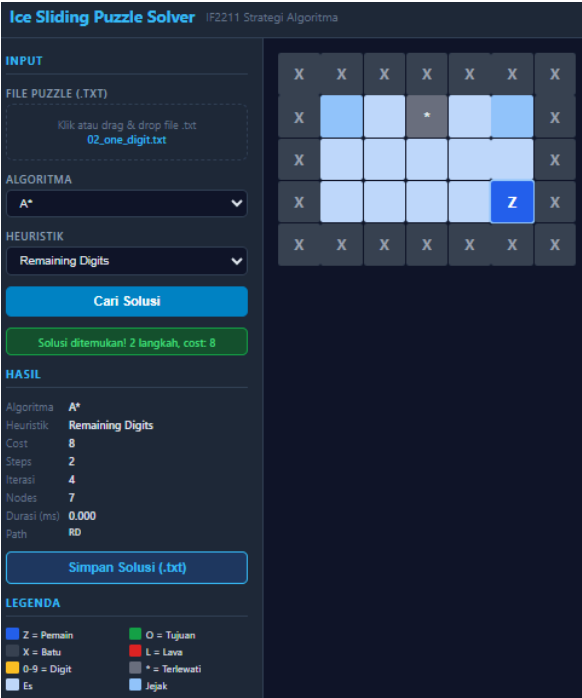
a. Tangkapan Layar Percobaan

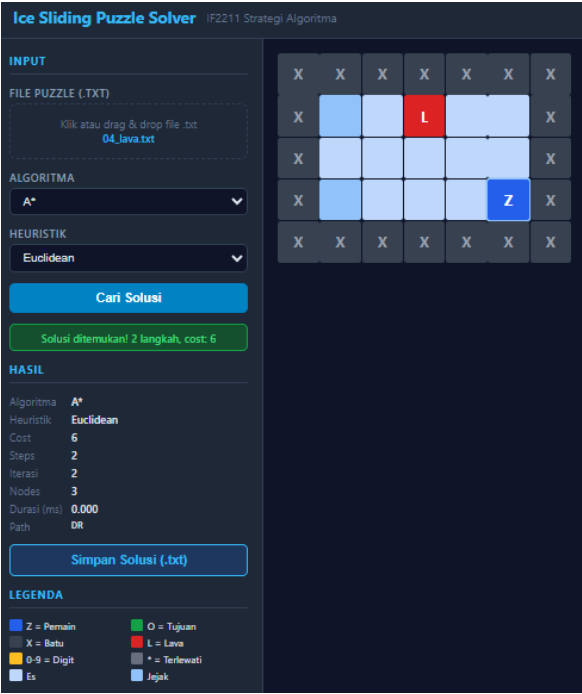
Percobaan 1 (03_multi_digit.txt)	
Input	Output
<pre> 6 8 XXXXXXXXX XZ*0X**X X*****X X**1***X X****O*X XXXXXXXXX 11111111 11121111 11111111 11141111 11111311 11111111 </pre>	 The screenshot shows the 'Ice Sliding Puzzle Solver' interface. The input puzzle is a 10x10 grid with various symbols. The algorithm is set to UCS (Uniform Cost Search) with a heuristic of Zero (UCS). The solution found is RDRULD, with a cost of 23 and 6 steps. The interface also displays the number of nodes (21) and the duration (0.000 ms).
Algoritma:	UCS
Heuristik :	Zero
Cost :	23
Steps :	6
Iterasi :	19
Nodes :	21
Durasi (ms) :	0.000
Path :	RDRULD

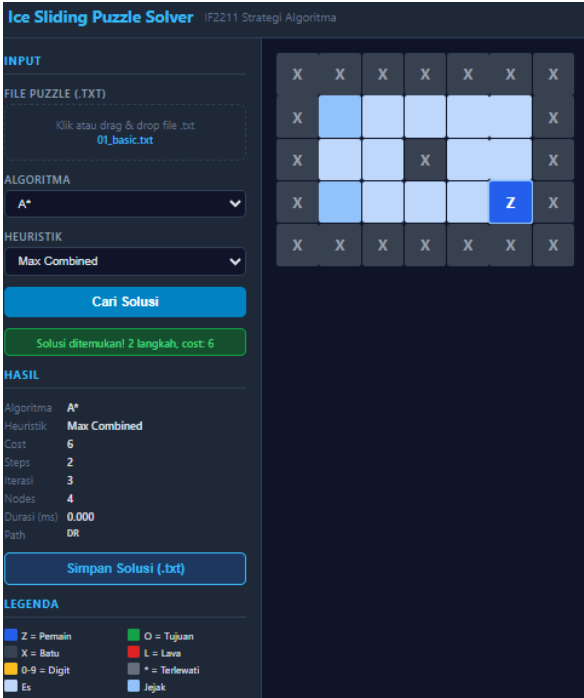
Percobaan 2 (03_multi_digit.txt)	
Input	Output
6 8 XXXXXXXXX XZ*0X**X X*****X X** ***X X****O*X XXXXXXXXX 11111111 11121111 11111111 11141111 11111311 11111111	
Algoritma:	GBFS
Heuristik :	Manhattan
Cost :	23
Steps :	6
Iterasi :	7
Nodes :	12
Durasi (ms) :	0.000
Path :	RDRULD

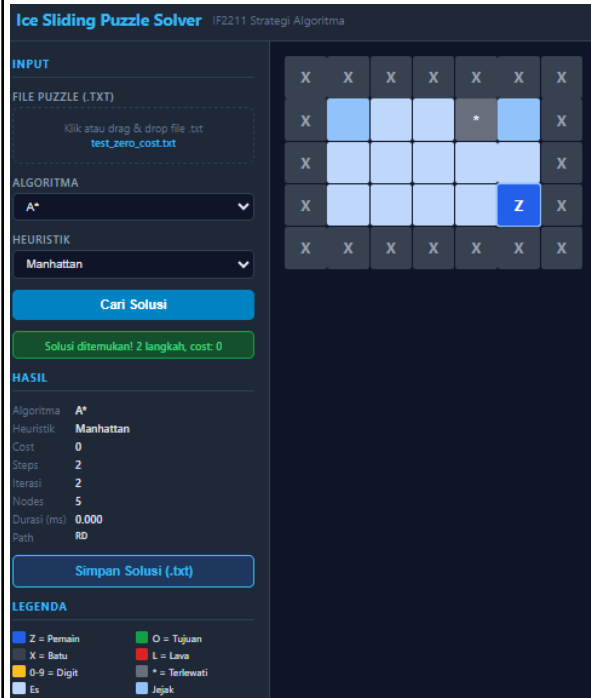
Percobaan 3 (03_multi_digit.txt)	
Input	Output
<pre> 6 8 XXXXXXXXX XZ*0X**X X*****X X**1***X X***O*X XXXXXXXXX 11111111 11121111 11111111 11141111 11111311 11111111 </pre>	 The screenshot shows the 'Ice Sliding Puzzle Solver' interface. The 'INPUT' section displays the puzzle grid from the text input. The 'ALGORITMA' section shows 'A*' selected. The 'HEURISTIK' section shows 'Manhattan' selected. A green button 'Cari Solusi' is visible. Below it, a green box states 'Solusi ditemukan! 6 langkah, cost 23'. The 'HASIL' section lists the solution details: Algorithm: A*, Heuristik: Manhattan, Cost: 23, Steps: 6, Iterasi: 17, Nodes: 20, Durasi (ms): 0.000, and Path: RDRULD. A 'Simpan Solusi (.txt)' button is at the bottom. A 'LEGENDA' section defines the symbols: Z = Pemain (blue square), X = Batu (grey square), 0-9 = Digit (yellow square), Es = (white square), O = Tujuan (green square), L = Lava (red square), * = Terlewat (grey square), and Jejak (blue square).
Algoritma:	A*
Heuristik :	Manhattan
Cost :	23
Steps :	6
Iterasi :	17
Nodes :	20
Durasi (ms) :	0.000
Path :	RDRULD

Percobaan 4 (03_multi_digit.txt)	
Input	Output
6 8 XXXXXXXXX XZ*0X**X X*****X X** ***X X****O*X XXXXXXXXX 11111111 11121111 11111111 11141111 11111311 11111111	
Algoritma:	IDA*
Heuristik :	Manhattan
Cost :	23
Steps :	6
Iterasi :	98
Nodes :	142
Durasi (ms) :	0.000
Path :	RDRULD

Percobaan 5 (02_one_digit.txt)	
Input	Output
<pre>5 7 XXXXXXXX XZ*0**X X*****X X****OX XXXXXXXX 1 1 1 1 1 1 1 1 2 1 3 1 1 1 1 1 5 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1</pre>	 The screenshot shows the 'Ice Sliding Puzzle Solver' application. The 'INPUT' section displays a 7x7 grid puzzle. The 'ALGORITMA' dropdown is set to 'A*'. The 'HEURISTIK' dropdown is set to 'Remaining Digits'. A green button 'Cari Solusi' is visible. Below it, a green box states 'Solusi ditemukan! 2 langkah, cost: 8'. The 'HASIL' section lists statistics: Algorithm: A*, Heuristik: Remaining Digits, Cost: 8, Steps: 2, Iterasi: 4, Nodes: 7, Durasi (ms): 0.000, Path: RD. A 'Simpan Solusi (.txt)' button is at the bottom. A 'LEGENDA' section defines symbols: Z = Pemain (blue square), X = Batu (grey square), 0-9 = Digit (yellow square), * = Terlewat (grey square), O = Tujuan (green square), L = Lava (red square), * = Terlewat (grey square), and Jejak (blue square).
Algoritma:	A*
Heuristik :	Remaining Digits
Cost :	8
Steps :	2
Iterasi :	4
Nodes :	7
Durasi (ms) :	0.000
Path :	RD

Percobaan 6 (04_lava.txt)	
Input	Output
<pre> 5 7 XXXXXXXX XZ*L**X X*****X X****OX XXXXXXXX 1111111 1119111 1111111 1111111 1111111 </pre>	 The screenshot shows the 'Ice Sliding Puzzle Solver' application. The input puzzle is a 7x7 grid with the following layout (rows from top to bottom): <pre> X X X X X X X X [] [] L [] [] X X [] [] [] [] [] X X [] [] [] [] [] Z X X X X X X X X </pre> The solver settings are: Algorithm: A*, Heuristic: Euclidean. The solution found is 2 steps with a cost of 6. The path is DR. The legend indicates: Z = Pemain (Player), X = Batu (Stone), 0-9 = Digit, Es = Ice, O = Tujuan (Goal), L = Lava, * = Terlewat (Overlooked), and Jarak = Distance.
Algoritma:	A*
Heuristik :	Euclidean
Cost :	6
Steps :	2
Iterasi :	2
Nodes :	3
Durasi (ms) :	0.000
Path :	DR

Percobaan 7 (01_basic.txt)	
Input	Output
<pre> 5 7 XXXXXXXX XZ****X X**X**X X****OX XXXXXXXX 11111111 11111111 11111111 11111111 11111111 </pre>	 The screenshot shows the 'Ice Sliding Puzzle Solver' interface. The input puzzle is a 7x7 grid with 'X' (Batu) and 'Z' (Permainan). The algorithm is set to A* with Max Combined heuristic. The solution found is 2 steps with a cost of 6. The path is DR.
Algoritma:	A*
Heuristik :	Max Combined
Cost :	6
Steps :	2
Iterasi :	3
Nodes :	4
Durasi (ms) :	0.000
Path :	DR

Percobaan 8 (test_zero_cost.txt)	
Input	Output
<pre> 5 7 XXXXXXXX XZ**0*X X*****X X****OX XXXXXXXX 0000000 0000000 0000000 0000000 0000000 0000000 </pre>	 The screenshot shows the 'Ice Sliding Puzzle Solver' interface. The input puzzle is a 7x7 grid with 'X' representing obstacles and '0' representing empty space. The goal is to move the player 'Z' to the target 'O'. The solver uses the A* algorithm with the Manhattan heuristic. The solution path is shown as 'RD', meaning the player moves Right then Down. The cost of the solution is 0.
Algoritma:	A*
Heuristik :	Manhattan
Cost :	0
Steps :	2
Iterasi :	2
Nodes :	5
Durasi (ms) :	0.000
Path :	RD

b. Analisis Hasil Percobaan

Berdasarkan percobaan yang telah dilakukan, berikut adalah analisis komparatif dari keempat algoritma (Uniform Cost Search, Greedy Best First Search, A* Search, dan Iterative Deepening A* Search) serta berbagai fungsi heuristic yang diimplementasikan.

- **Perbandingan Kinerja Algoritma**

Untuk membandingkan *space complexity* dan *time complexity* secara adil, keempat algoritma diuji menggunakan konfigurasi papan yang sama dengan heuristic Manhattan (untuk algoritma pencarian informed). Berdasarkan data pada hasil pengujian, analisis untuk tiap algoritma dijabarkan sebagai berikut:

1. **Validitas Solusi Optimal:** Keempat algoritma berhasil mendapatkan solusi optimal dengan jalur dan cost (23) yang seragam. Hal ini membuktikan bahwa A* Search dan IDA* Search telah mengimplementasikan heuristic yang admissible, sehingga algoritma ini dapat selalu mendapatkan solusi yang optimal sebagaimana halnya yang dijamin oleh Uniform Cost Search.
2. **Perbandingan A* Search dan Uniform Cost Search:** Dari pengamatan, didapatkan bahwa UCS membangkit simpul yang lebih banyak jika dibandingkan dengan A* Search. Uniform Cost Search memperluas simpul secara seragam berdasarkan biaya kumulatif ($g(n)$). Di sisi lain, A* Search memperluas simpul berdasarkan f-value di mana heuristic yang baik dapat memangkas ruang pencarian yang tidak perlu, sehingga algoritma menjadi semakin efisien dan menghemat ruang memori.
3. **Agresivitas Greedy Best First Search:** GBFS memperluas simpul paling sedikit dan memiliki runtime yang sangat cepat (7 iterasi). Algoritma ini mengeksplorasi ruang pencarian secara greedy dengan hanya mempertimbangkan nilai heuristic $h(n)$ menuju solusi. Pada pengujian ini, ia berhasil mendapatkan solusi optimal. Namun, pendekatan yang selalu mengambil estimasi terendah pada saat itu tanpa melihat ke belakang dapat membuat GBFS terjebak dalam solusi sub-optimal pada ruang status yang lebih kompleks.
4. **Karakteristik Iterative Deepening A* (IDA*):** Terdapat hal yang unik terjadi pada implementasi IDA*. Algoritma ini membangkit simpul yang jauh lebih banyak (142 simpul) di saat algoritma lain menghemat ruang pencarian. Hal ini terjadi karena IDA* membatasi ruang pencarian pada suatu fase dengan nilai f-limit. Alih-alih menyimpan seluruh *frontier*

dalam memori seperti A* Search, IDA* mengulang kembali pencarian dari awal tingkat per fase setiap kali batas cost terlampaui. *Overhead* iterasi ini adalah harga yang dibayar untuk mengurangi *overhead* memori dari *priority queue*.

- **Tabel Space Complexity dan Time Complexity Algoritma Pathfinding**

Algoritma	Time Complexity	Space Complexity
UCS	$O(b^d)$	$O(b^d)$
GBFS	$O(b^m)$	$O(b^m)$
A*	$O(b^d)$	$O(b^d)$
IDA*	$O(b^d)$	$O(d)$

- **Pengujian Heuristic Alternatif dan Ruang Status Khusus**

Analisis lebih lanjut dilakukan untuk mengevaluasi ketahanan program dan variasi heuristic pada berbagai kondisi kasus uji:

1. **Performa Heuristic yang Informatif:** Penggunaan heuristic yang lebih baik dan informatif, seperti Euclidean dan Max Combined pada kasus 01_basic.txt dan 02_one_digit.txt, terbukti mampu memangkas ruang pencarian dengan sangat agresif. Heuristic yang baik berhasil mempercepat *runtime* hingga solusi ditemukan hanya dalam 2 hingga 4 iterasi saja tanpa mengorbankan optimalitas solusi.
2. **Penanganan Rintangan dan Status Kematian (04_lava.txt):** Dalam implementasinya, algoritma dapat dengan cerdas tidak memperluas simpul yang berisi Lava (L) atau sisi papan tanpa batu. Algoritma selalu mempertimbangkan kondisi ini sebagai simpul buntu (*dead end*) sehingga solusi yang dihasilkan selalu valid dan terhindar dari kondisi *Game Over*.
3. **Pencarian pada Graf dengan Bobot Nol (test_zero_cost.txt):** Algoritma tetap mendapatkan solusi secara optimal dan tidak terjebak dalam siklus (*cycle*) saat memperluas simpul-simpul yang memiliki bobot langkah 0,

serta mengkalkulasikan cost akhir dengan akurat. Penanganan status yang telah dikunjungi (*visited states*) berjalan baik dalam mencegah algoritma terjebak pada jalan buntu.

F. *Link Repository*

Kode program lengkap beserta instruksi penggunaan dapat diakses melalui *link repository* berikut: [NazwanSM/Tucil3_18223066_18223096](#)

G. Surat Pernyataan

Tugas ini disusun sepenuhnya tanpa bantuan kecerdasan buatan (*Generative AI*), melainkan hasil pemikiran dan analisis mandiri.



Nazwan Siddqi Muttaqin



Matthew Sebastian Kurniawan

H. Lampiran

No	Poin	Ya	Tidak
1	Program berhasil di kompilasi tanpa kesalahan	✓	
2	Program berhasil dijalankan	✓	
3	Solusi yang diberikan program benar dan mematuhi aturan permainan	✓	

4	Program dapat membaca masukan berkas .txt serta menyimpan solusi dalam berkas .txt	✓	
5	Program memiliki Graphical User Interface (GUI)	✓	
6	Ada algoritma <i>pathfinding</i> alternatif selain dari spesifikasi wajib (Algoritma X dan Algoritma Y)	✓	
7	Ada tambahkan dua alternatif heuristik selain yang utama (Heuristik X dan Heuristik Y)	✓	

I. Referensi

Rinaldi Munir. (2026). "Route Planning (Bagian 1)." Diakses pada 3 Mei 2026.
[https://informatika.stei.itb.ac.id/~rinaldi.munir/Stmik/2025-2026/21-Route-Planning-\(2026\)-Bagian1.pdf](https://informatika.stei.itb.ac.id/~rinaldi.munir/Stmik/2025-2026/21-Route-Planning-(2026)-Bagian1.pdf)

Rinaldi Munir. (2026). "Route Planning (Bagian 2)." Diakses pada 3 Mei 2026.
[https://informatika.stei.itb.ac.id/~rinaldi.munir/Stmik/2025-2026/22-Route-Planning-\(2026\)-Bagian2.pdf](https://informatika.stei.itb.ac.id/~rinaldi.munir/Stmik/2025-2026/22-Route-Planning-(2026)-Bagian2.pdf)